

腾讯安全云鼎实验室

TENCENT SECURITY YUNDING LAB

云安全技术手册



铸力为鼎，护云安全

2021年9月出版

腾讯云计算〔北京〕有限公司

目录

Contents

01. 腾讯安全云鼎实验室介绍

1.1. 关于云鼎实验室	07
1.2. 业界影响力及荣誉	08

02. 云原生安全

2.1. 云原生安全理念	10
2.1.1. 云计算成为重要基础设施，安全性备受关注	10
2.1.2. 云原生安全理念兴起	10
2.1.3. 云原生安全定义	11
2.2. 云原生安全趋势与展望	11
2.3. 腾讯云原生安全体系图谱	12

03. 云安全攻防矩阵

3.1. 云安全攻防矩阵	14
--------------	----

04. 云安全攻防实践

4.1. 容器安全	16
4.1.1. 前言	16
4.1.2. 容器发展历程概览	16
4.1.3. 风险面概览	18
4.1.4. 容器产品中的传统攻击面	19
4.1.5. API	19
4.1.6. 下发过程	19
4.1.7. 执行环境	19
4.1.8. 总结	19
4.2. RunC TOCTOU 逃逸 CVE-2021-30465 分析	20
4.2.1. 背景	20
4.2.2. 漏洞分析	21
4.2.3. 补丁分析	24
4.2.4. POC 分析	25
4.2.5. 总结	26
4.3. 浅谈云上攻防-元数据服务带来的安全挑战	26
4.3.1. 前言	26
4.3.2. 元数据服务以及角色介绍	26
4.3.3. 针对元数据服务的攻击	28
4.3.4. 元数据安全性改进	33
4.3.4. 元数据服务更多安全隐患	34
4.4. SSRF 漏洞对云原生环境带来的威胁	35
4.4.1. 前言	35
4.4.2. SSRF 与云环境结合带来的严重影响	36
4.4.3. 自动化收敛云上 SSRF 漏洞	40
4.4.4. 写在最后	40
4.5. 国内首个对象存储 ATT&CK 攻防矩阵	40
4.5.1. 对象存储服务攻防矩阵概览	41
4.5.2. 初始访问	41
4.5.3. 执行	43
4.5.4. 持久化	43

4.5.5. 权限提升	43
4.5.6. 窃取凭证	44
4.5.7. 横向移动	44
4.5.8. 影响	45
4.6. 云原生时代，腾讯如何做好容器安全体系建设？	46
4.6.1. 背景	46
4.6.2. 容器安全体系设计四大原则	46
4.6.3. 全方位层次化的云原生容器安全体系框架	47
4.6.4. 安全能力全面原生化	48
4.6.5. 安全防护全生命周期	49
4.6.6. 安全威胁全面可观测	49
4.6.7. 安全责任共担	50
4.6.8. 总结	50

腾讯安全 云鼎实验室介绍

Tencent
Security
YUNDING Lab
Introduction

1.1 关于云鼎实验室

腾讯安全云鼎实验室专注云安全技术研究和云安全产品创新工作；负责腾讯云安全架构设计、腾讯云安全防护和运营工作；通过攻防对抗、合规审计搭建管控体系，提升腾讯云整体安全能力。

同时，基于前沿领域的研究和探索，发现前沿技术中可能存在的安全问题，守护政府及企业的数据、系统、业务安全，运用前沿技术解决安全问题，以紧贴业务安全的最佳实践为产业数字化升级保驾护航。



图1-业务与技术蓝图

1.2 业界影响力及荣誉

云鼎实验室积极参与云原生安全、研发运维安全、数据安全等前沿课题研究，投入专家资源，把自身的技术积累和创新，贡献到云计算建设和标准制定中，共参与了10多项国内标准/国际标准的研制。

云鼎将持续深化云计算运维、云原生安全、数据安全、密钥安全等技术规范化、标准化地向前发展，与生态伙伴共同推动我国云计算技术产业蓬勃发展，为“新基建”等国家发展战略贡献力量。

标准名称	标准机构	标准类型
《研发运营一体化（DevOps）能力成熟度模型》	IITU-T	国际标准
《面向云计算的可信研发运营安全能力成熟度模型》	信通院	国家标准
《研发运营安全白皮书》	信通院	行业报告
《网络安全标准实践指南——远程办公安全防护》	信安标委	国家标准
《基于容器的平台安全能力要求》	信通院	国家标准
《可信云-容器安全能力认证》	信通院	行业标准
《等保2.0容器安全标准》	公安部	行业标准
《云原生架构安全白皮书》	信通院	行业报告
《“云”原生安全白皮书》	信通院	行业报告
《物联网及只会可持续城市数据中台框架》	ITU-T	国际标准
《信息技术-智慧城市-公共卫生应急城市服务平台》	IEC/ISO	国际标准
《2020 DevSecOps 企业实践白皮书》	Freebuf	行业报告
《腾讯密码算法试用与密钥安全管理规范》	腾讯	企业标准
《静态应用程序安全测试(SAST)工具 能力要求》	信通院	行业标准
《交互式应用程序安全测试(IAST)工具能力要求》	信通院	行业标准
《腾讯觅影构建患者为中心安全链接-医疗影像云应用与安全能力评估白皮书》	腾讯	企业标准
《远程医疗线上应用服务系统安全测评规范 第1部分：渗透测试》	腾讯	企业标准

图2-云鼎参与制定标准

2020年9月，知识产权权威机构发布专利分析报告，报告显示腾讯云安全技术专利申请量、专利授权量、专利价值稳居行业第一。同时，在数据安全和攻防两个细分领域的专利申请量也是行业第一，进一步证明了云鼎实验室在该领域的创新实力。

云鼎实验室致力于走技术创新的路线，把知识产权贯穿到研发的全生命周期，在数据安全（如数据加密、密钥管理、云密码机）、攻防（如入侵检测、漏洞检索、人脸黑产）、安全运营（如漏洞情报、MSS服务）、计算安全（如容器安全、可信计算）等领域申请了若干项发明专利，截止2021年8月，共有80余件专利递交在审阶段，16件专利获得授权。

云原生安全

Cloud Native Security

2.1 云原生安全理念

2.1.1 云计算成为重要基础设施，安全性备受关注

云计算成为底层基础设施，帮助企业实现数字化转型。近年来，随着 5G、云计算、大数据、人工智能等新技术的持续发展，以数字化和智能化等为核心内容的第四次工业革命浪潮已汹涌而至。数字化变革将对经济和社会产生深远影响，越来越多的国家把发展数字经济 作为推动经济增长的重要途径。

中共十九大提出建设“数字中国”的宏伟蓝图，同时要求推动互联网、大数据、人工智能和实体经济深度融合。近两年，随着国家和各地相继发布推动 数字化转型的政策文件，数字时代的到来，以及数字泛在化已经成为了一种必然的趋势。数 字技术将深入改造企业生产、研发、销售、管理等各个环节，对于海量数据的采集、分析、挖掘、利用将为企业决 策与运营提供重要帮助，数字资产将成为企业组织最重要的资产之一。

数字化时代，数据信息在企业内部各业务模块、环节顺畅传递是企业实现数字化转型的必要条件。相比传统的本地部署模式，云计算具有快速部署、即接即用、弹性扩容、按需付费等特点，可以有效降低企业的使用成本，提升使用效率。针对海量数据处理所需要的巨大 算力，云计算依托低成本、快速部署、弹性扩容等特点，可以随时提供强大的弹性伸缩的计算资源。

目前，云计算已经成为底层基础设施，企业实现数字化转型，离不开云计算等技术 的发展，云计算、大数据等技术的大规模推广使用不开云计算等技术的发展，云计算、大数 据等技术的大规模推广使用有助于实现有助于实现企业内部的数字化改造，提升数据的运转效率，帮助企业企业内部的数字化改造，提升数据的运转效率，帮助企业最终实现数字化转型。最终实现数字化转型。

2.1.2. 云原生安全理念兴起

为缓解传统安全防护建设中存在的痛点，促进云计算成为更加安全可信的信息基础设施，助力云客户更加安全地使用云计算，云原生安全理念逐渐兴起。国内外第三方组织、服务商纷纷提出以原生为核心构建和发展云安全。例如Gartner提倡以云原生思维建设云安全体系。

基于云原生思维Gartner提出的云安全体系覆盖八方面，如图 5 所示。其中，基础设施配置、身份和访问管理两部分由云服务商作为基础能力提供，其它六部分需基于安全产品实现，包 括持续的云安全态势管理，全方位的可视化、日志、审计和评估，工作负载安全，应用、PaaS 和 API 安全，扩展的数据保护，云威胁检测。



图3-Gartner 云安全体系

2.1.3. 云原生安全定义

国内外各组织、企业对云原生安全理念的解釋略有差异，结合我国产业现状与痛点来看 云原生安全指云平台安全原生化和云安全产品原生化。安全原生化的云平台，一方面通过云 计算特性帮助用户规避部分安全风险，另一方面能够将安全融入从设计到运营的整个过程中，向用户交付更安全的云服务；原生化的云安全产品能够内嵌融合于云平台，解决用户云 计算环境和传统安全架构割裂的痛点。

作为一个全新的安全理念，云原生安全旨在将安全与云计算深度融合，推动云服务商提 供更安全的云服务，帮助云计算客户更安全的上云。

2.2 云原生安全趋势与展望

随着云计算安全需求愈加迫切，云安全技术与产业不断发展，云原生安全体系也将更加成熟健全，具体表现为：

1. 云服务商与安全厂商联合建设云原生安全生态

随着云计算的发展，客户类型日益多样，不同用户群体面临不同安全场景，安全需求差异大。同时，网络安全环境日益严峻，云安全所涉范围越来越广，构成愈加复杂。仅凭一家厂商的技术与管理能力，云原生安全建设难以面面俱到，一些领域暴露短板。

未来，云服务商与安全厂商势必加强深度合作，结合双方在技术研究、人才储备、产品应用等方面的积累和经验 建设一个更加完善、开放的云原生安全生态。

2. 产品趋于整合，形成综合的云原生安全解决方案

目前，云原生安全已建立了包括云主机安全、敏感数据处理、DDoS防护、云防火墙、安全运营中心等在内的较为丰富的产品体系单个产品可以缓解用户在某些场景下的安全问题。但随着用户对云上安全体系建设全面化、完善化的不断追求，未来，云原生安全产品将趋于整合，各产品之间有效联动增加，以综合的安全解决方案的形式交付给用户，进一步降低用户安全建设复杂度，助力用户更加便捷、快速的构建云上安全体系。

3. 深耕行业，助力传统行业云上安全体系建设

政务、金融、工业、农业等行业业务场景各有特色，上云后安全需求和痛点有差异，如政务云内跨业务区的数据同步和交换对安全性的要求，金融云面临高监管要求等。未来，云原生安全将更加聚焦行业，结合行业安全需求，精细化发展云原生安全产品功能。

2.3 腾讯云原生安全体系图谱

围绕云原生安全，目前腾讯安全已搭建了包含安全治理、数据安全、应用安全、计算安全、网络安全等五个领域的完备云原生安全防护体系，将云原生的安全产品开放给腾讯云上的客户，让每一位客户都能使用腾讯级的安全产品。

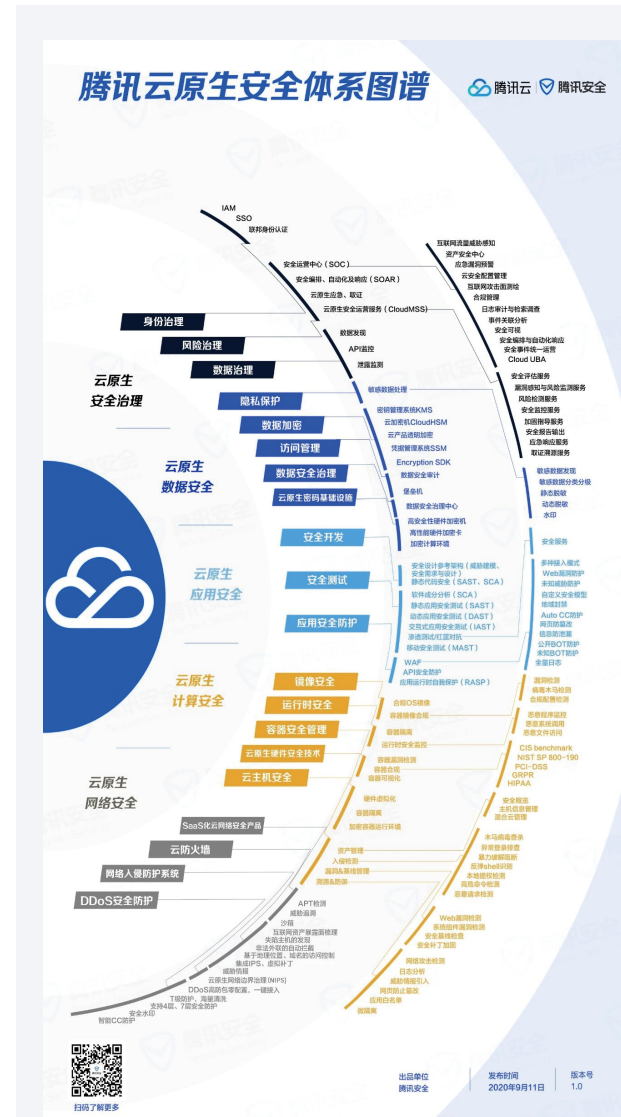


图4-腾讯云原生安全体系图谱

在持续发力云原生安全研究的同时，腾讯云原生安全体系已经在实践中收获成果，护航腾讯自身业务及腾讯云用户的安全。

1.云原生安全体系支撑腾讯会议快速扩容

疫情期间，腾讯会议用户规模高速增长，产品迅速扩容的同时，也对安全防护提出了更高的要求。基于这一诉求，腾讯安全在腾讯会议产品设计之初就将安全体系内置产品之中，使安全能力能够伴随产品迭代升级，最终协同腾讯会议团队取得 8 天云主机扩容 10 万台、上线 2 个月日活突破 1000 多万、100 天更新 20 个版本等成绩，保障了腾讯会议在复工复产期间的大规模应用。

2.打造云原生免费合规镜像，助力企业云服务器一键过等保

依靠云原生安全体系防护自家产品的同时，腾讯安全还将这种能力持续开放给云上用户。为帮助云上用户快速解决云服务器合规问题，腾讯安全今年推出一套云原生的免费合规镜像，用户仅需在官网上进行简单操作，便能获得一套符合等保要求的云主机。

3.携手Github检测API密钥泄露，累计挽回数亿元潜在损失

此外，腾讯安全还与Github官方达成合作，帮助云上用户检测到数千次 API 密钥泄露 事件并进行告警，快速响应协助用户修复，通过KMS白盒加密的方式对API密钥进行加密。累计帮助云上用户挽回数亿元潜在经济损失。

在行业落地之外，腾讯云原生安全产品也得到了国际权威机构的认可。今年，腾讯云主机安全服务成功入选 Gartner CWPP 全球市场指南，成为中国首家获得指南推荐的全功能、多平台CWPP全球供应商，为百万级主机安全保驾护航。随着虚拟机、云主机、容器等技术 的不断落地，腾讯安全正将云主机安全、云防火墙、云 SOC共同打造成一个更加完善的云原生安全防护体系，持续保障云上用户安全。

随着云计算技术与产业的不断发展，更多的企业采用云计算实现数字化转型。虽然云计算拥有着很多优势，但也带来了全新的安全挑战。

MITRE ATT&CK® 框架是一种用来描述攻击阶段与每个阶段所使用的攻击技术与手段的知识框架，安全人员可以通过此框架有效的了解当前环境中所面临的攻击面，并以此制定监测手段用以发现风险。

腾讯安全云鼎实验室针对云上安全所面临的威胁以及攻击技术进行整理，推出云上安全攻击矩阵，该矩阵共分为初始访问、执行、持久化、权限提升、防御绕过、窃取凭据、探测、横向运动、影响这九大阶段，每个都包含了多种用以实现此阶段能力的攻击技术。

云安全 攻防矩阵

Cloud Security Offensive And defensive Matrix

云安全攻防矩阵																						
初始访问	执行	持久化	权限提升	防御绕过	窃取凭据	探测	横向移动	影响														
云平台账户泄露	使用云API执行命令	在存储对象中植入后门	通过Write acl提权	通过代理访问	获取配置文件中的应用凭证	探测云厂商服务接口	窃取凭据访问云服务	数据泄漏														
云API密钥泄露	使用云厂商工具执行	云服务器实例中部署远控客户端	通过访问管理提权	禁用日志记录	云服务凭证泄露	内网扫描	窃取用户账号攻击其他应用	资源劫持														
实例登录信息泄露	登录云服务器实例执行指令	在userdata中添加后门	修改policy策略	监控区域外创建实例	获取服务器实例登录账号	探测Kubernetes Dashboard	使用实例账号爆破	网络逃逸														
账户劫持	写入userdata执行命令	给现有用户分配额外的密钥创建新实例	sqlserver提权	关闭全局服务事件监视	元数据服务获取角色临时凭据	探测 Kubernetes APIServer	通过控制台权限横向移动	拒绝服务														
网络钓鱼	利用后门文件执行指令	给现有用户分配额外的密钥	利用操作系统漏洞进行提权	关闭主机安全产品	获取K8s configfile	内网扫描	窃取角色临时凭据横向访问	数据丢失														
应用程序漏洞	利用数据库应用执行命令	建立辅助账号登录	利用特权容器提权	日志清理	元数据服务获取角色临时凭据	通过K8s Dashboard横向移动	通过漏洞逃逸到宿主机	服务滥用														
使用恶意或存在漏洞的自定义镜像	利用SSH服务进入实例执行	通过容器逃逸向宿主机写入文件	利用错误配置提权	关闭容器安全产品	用户账号数据泄露		通过配置错误逃逸到宿主机	权限接管														
应用程序漏洞	利用远程代码执行漏洞执行	在自定义镜像库中镜像植入后门	利用操作系统漏洞提权	常规Pod中植入后门	该颜色代表：cos存储攻击方式		通过K8s Dashboard横向移动	窃取用户账号攻击其他应用														
使用恶意或存在漏洞的自定义镜像	部署后门容器	K8s 定时任务	利用Kubernetes漏洞进行提权	日志清理			窃取角色临时凭据访问云服务															
暴露Kubernetes控制面板	进入容器执行	该颜色代表：容器攻击方式	利用Docker漏洞提权	该颜色代表：云服务器攻击方式			窃取用户账号攻击其他应用															
kubeconfig文件泄露	利用SSH服务进入容器执行		该颜色代表：共同攻击方式				该颜色代表：cos存储攻击方式															
对象存储SDK泄露	利用远程代码执行漏洞执行																					
存储桶工具配置文件泄露	该颜色代表：cos存储攻击方式																					
前端直传功能凭取凭据																						
实例元数据服务未授权访问																						

该颜色代表：共同攻击方式

该颜色代表：云服务器攻击方式

该颜色代表：容器攻击方式

该颜色代表：cos存储攻击方式

图5-云安全攻防矩阵

云安全 攻防实践

Cloud Security Offensive And defensive Practices

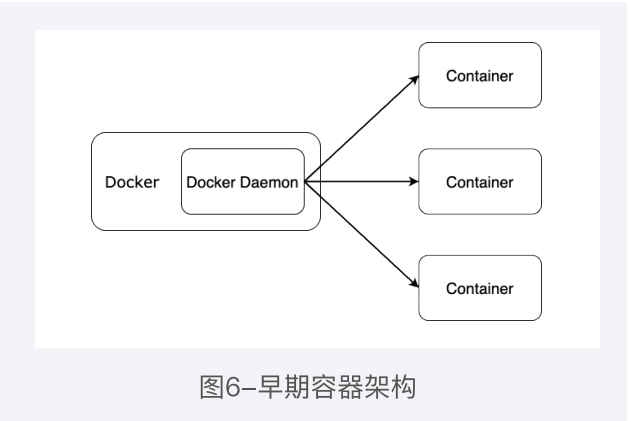
4.1 容器安全

4.1.1. 前言

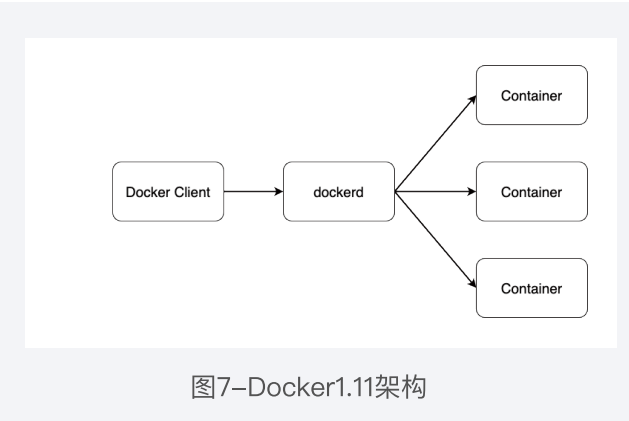
2020 年是云原生技术迎来爆炸增长的一年，并且未来仍延续高速增长态势。容器作为 云原生核心基础设施，在云厂商的视角中，容器不仅是业务运行的基石，同时容器也将作为 商业化产品形态对客户提供更强的容器能力，其形态主要表现分为 Caas （Container as a service）与 Kaas（Kubernetes as a service）两类，其是否安全直接关系到下游产品安全以及客户产品安全，故容器安全可谓云安全的核心板块。

跟普通容器使用者不同是的，作为云服务提供商，容器相关产品的资源下发流程相对复杂，在一些场景下使用客户的计算资源进行搭建。整个流程中涉及资源下发、环境初始化、 相关服务安装、证书下发、建立链接等多个步骤，不同于以往的产品场景，攻击者基于此可直接与 API 服务器或某些系统内公共服务直接通信，自由度非常高，稍有不慎，可能导致系统与租户的边界被打破，直接危害云基础安全。

4.1.2. 容器发展历程概览



在很久之前（Docker < 1.11），Docker 只是简单的 C/S 架构，Docker 集成了 DockerDaemon 用于容器的管理。



这种模式简单易行，但扩展性不强，在 Docker 1.11 中，将 docker daemon 抽出为独立的二进制文件 dockerd。

为了让容器生态更加开放，Linux基金会发起OCI（Open Container Initiative），目标是标准化容器格式和运行时，其中一个重要产物就是CRI（Container Runtime Interface），抽象了容器运行时接口，使得上层调控容器更加便捷。containerd和runc都是其中代表产物，从dockerd中再剥离出containerd，向上提供rpc接口，再通过containerd去管理runc。containerd在初期也是直接对runc进行管理，但为了解决containerd进行升级等操作时会造成不可用的问题，containerd再拆出containerd-shim，独立对接runc。containerd从Runtime、Distribution、Bundle维度提供容器全生命周期的管理能力，runc专注于Runtime。

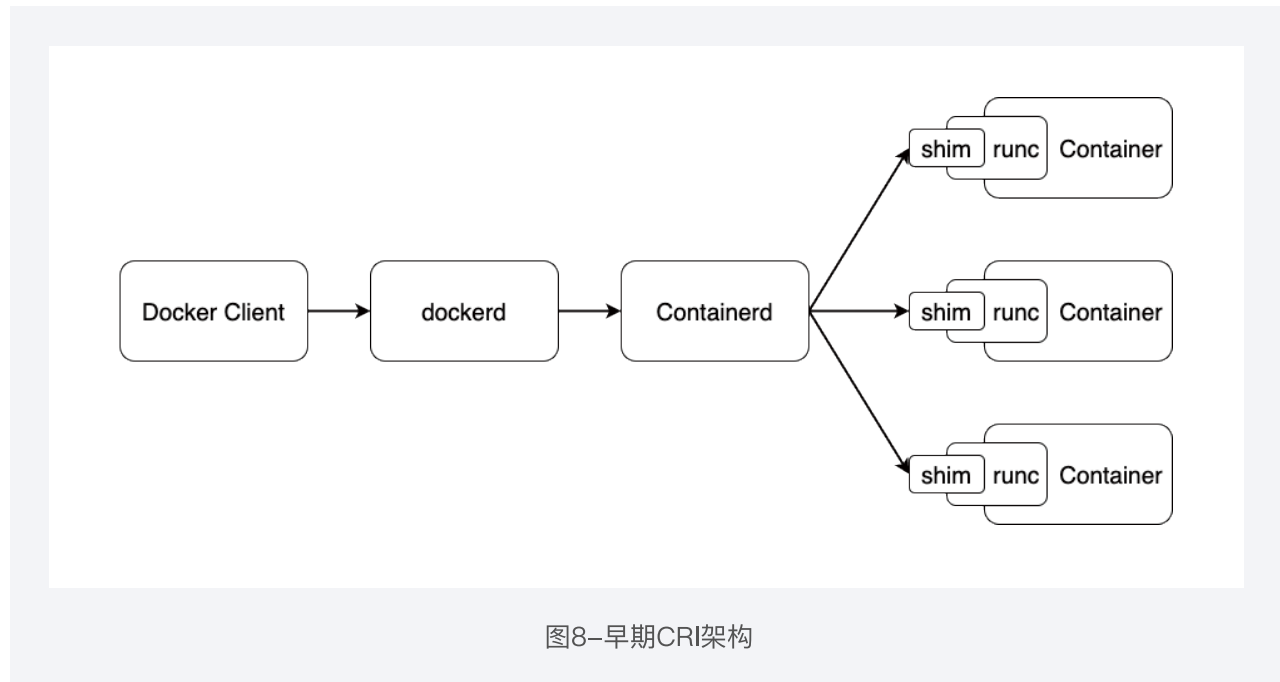


图8-早期CRI架构

为了让容器生态更加开放，Linux基金会发起OCI（Open Container Initiative），目标是标准化容器格式和运行时，其中一个重要产物就是CRI（Container Runtime Interface），抽象了容器运行时接口，使得上层调控容器更加便捷。containerd和runc都是其中代表产物，从dockerd中再剥离出containerd，向上提供rpc接口，再通过containerd去管理runc。containerd在初期也是直接对runc进行管理，但为了解决containerd进行升级等操作时会造成不可用的问题，containerd再拆出containerd-shim，独立对接runc。containerd从Runtime、Distribution、Bundle维度提供容器全生命周期的管理能力，runc专注于Runtime。

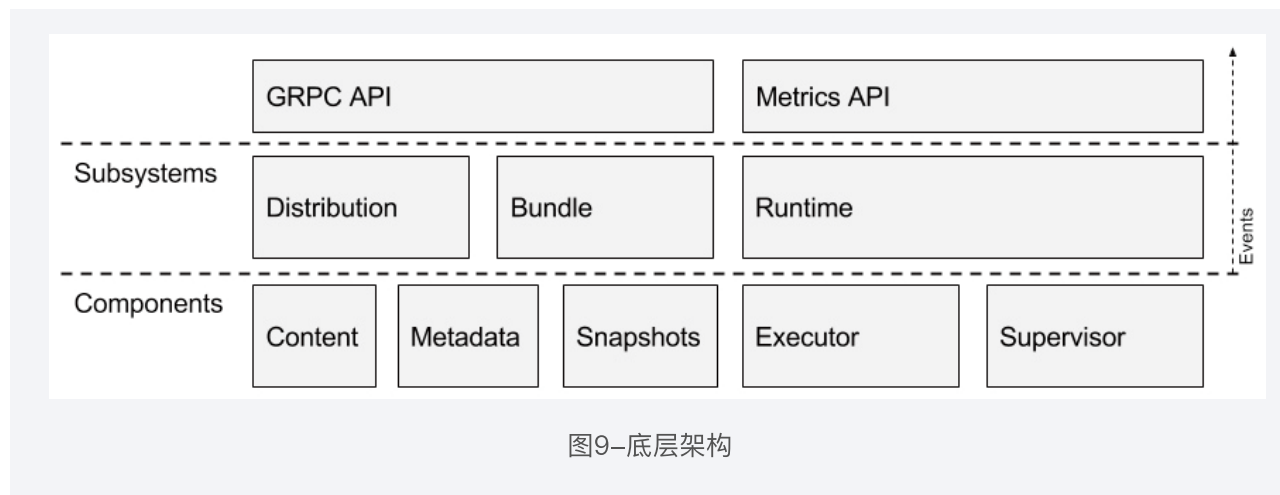


图9-底层架构

在上述基础上，为Kubernetes、Docker Swarm等编排系统提供了坚实的基础，使得Kubernetes等编排系统可以通过rpc直接对Containerd进行调控，但不是自己的东西用起来终究不爽，于是Kubernetes联合几个巨头搞了自己的CRI实现，CRI-O，所以目前整体架构如下图12所示：

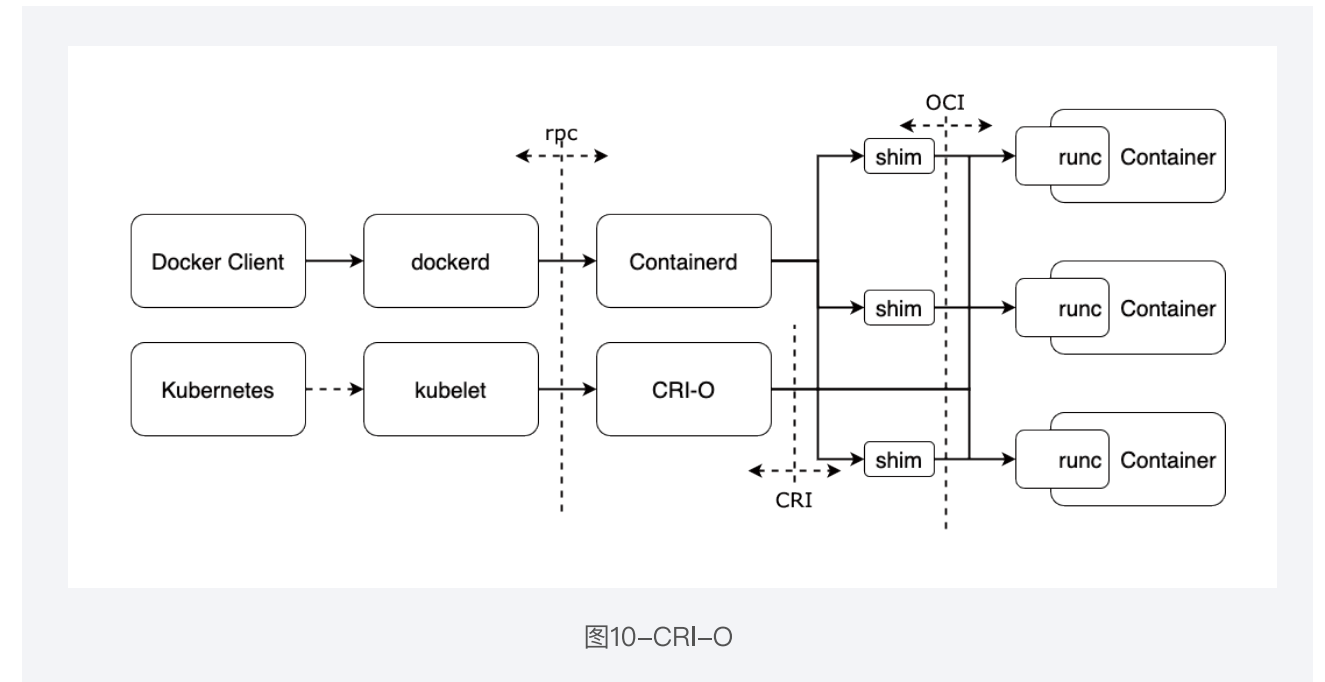


图10-CRI-O

4.1.3. 风险面概览

容器使用中可能导致的风险面已经相对比较成熟，各个方面都有了比较详细的讨论。

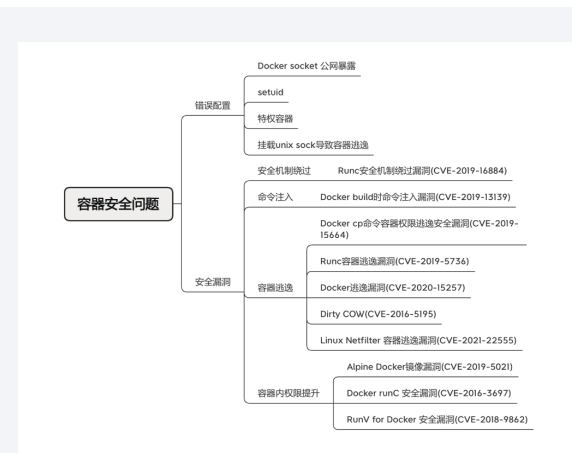


图11-容器安全问题

- 从目前容器相关的漏洞上可看出，利用Linux内核进行容器逃逸的漏洞相对比较难进行发现，但利用容器相关安全机制和组件特性进行逃逸的漏洞呈现一个增长的态势。传统的内核逃逸提权依然存在，随着对容器的深入研究，containerd、runc等组件进行逃逸的情况也越来越多。

在容器产品漏洞挖掘的角度中，情况将又发生了一些变化。在传统漏洞维度上，又引入了新的攻击面。

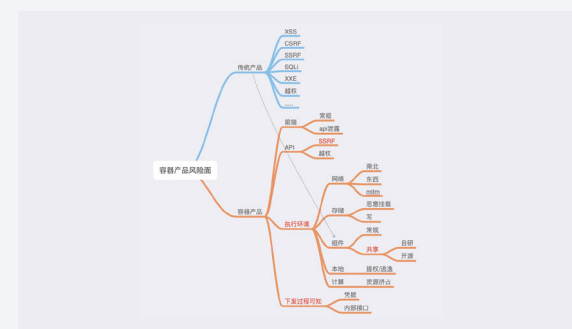


图12-容器产品风险面

- 在传统产品中，虽然产品业务丰富多样，但产品形态上相对稳定，所暴露的风险面也比较固定，安全人员在相关领域的研究也比较深入。在容器产品中发生了较大的改变，除去传统的WEB层面上的交互之外，还提供了执行环境，在该环境中，网络、组件、存储、计算等层面均暴露出了新的攻击面。

4.1.4. 容器产品中的传统攻击面

在新场景下，依然存在这一些老问题。在执行环境中，可访问的网络（对VPC暴露）组件、本地组件存在两种情况，一种是每个用户所独立占有的，组件的生命周期伴依赖资源的生命周期。另外一种情况是，多个租户访问共同的一个组件，比如镜像仓库或agent server等服务，在这种情况下传统漏洞依然会存在于这种情况中。

4.1.5. API

当前云环境中提供了多种多样的API，方便开发者进行调用。除去传统漏洞挖掘中API所面临的风险，新场景中SSRF和越权的危害将是较高的。SSRF将打破产品边界隔离，将攻击流量转变为信任的流量，将能够调用很多外部无法进行调用的接口。同时，云API将提供资源的管理权限，越权将导致打破多租户间的隔离，敏感接口的越权可直接导致其他客户的云资源被攻击者入侵。

4.1.6. 下发过程

容器产品（Kaas类）将在用户自己的计算资源中进行部署的构建，其过程在可以通过某种方式进行捕获。该过程中可能存在某些敏感的凭据或高权限凭据，该过程泄露可导致控制面某些服务被入侵。在资源安装过程中，存在内部服务接口暴露的情况，内部服务接口的安全性较弱，在传统漏洞防护和权限控制上可能存在一定的风险。

4.1.7. 执行环境

容器的多租户执行环境中网络隔离相对较弱，东西向的网络逃逸或中间人攻击将导致其他租户受影响，在某些场景下甚至能导致其他租户的数据被盗取。南北向的流量逃逸将影响控制面中某些内部的控制接口被访问，此类控制接口大多具有极高的权限，可导致资源直接被入侵。

在存储的维度中，恶意挂载将导致宿主机的硬盘部分或全部被攻击者读取到，硬盘中将存储较多的敏感信息。同样，恶意写入可能导致更大的威胁，恶意写入存在时，绝大部分情况会导致逃逸。

多租户的前提下，每个租户的计算资源限额应被严格限制，如被打破，将挤占在同一宿主机上的其他租户的计算资源，导致无法正常进行使用。

4.1.8. 总结

容器产品安全不同于传统意义上的容器安全，伴随着对容器/集群安全的研究深入，容器产品的安全也将迈入一个全新的阶段。

4.2 RunC TOCTOU逃逸 CVE-2021-30465分析

4.2.1. 背景

国外安全研究员champtar在日常使用中发现Kubernetes tmpfs挂载存在逃逸现象，研究后发现runC存在条件竞争漏洞，可以导致挂载逃逸。关于条件竞争TOCTOU和一些linux文件基础知识可见文章《初探文件路径条件竞争 – TOCTOU&CVE-2019-18276》。

CVE-2021-30465在Redteam的研究者视角中比较鸡肋，因为需要K8S批量创建POD的权限。但在产品安全的视角恰恰相反，针对Caas（Container as a service）类产品，用户/租户拥有批量创建POD权限，利用挂载逃逸可打破租户间隔离，同时读取Host层面某些敏感数据，危害性极大。

1. Namespace

Namespace是linux控制系统资源的抽象层，将不同的进程放置入不同的Namespace将获得不同的资源视角，该项技术是容器实现的基础。linux提供8种不同的Namespace以提供不同维度的隔离能力，分别是：1. Cgroup；2. IPC；3. Network；4. Mount；5. PID；6. Time；7. User；8. UTS。

其中，Cgroup和Mount Namespace是最常接触的，在容器挂载相关能力均通过Mount Namespace进行实现。Namespace的使用主要通过clone和unshare 两个方法实现，其中clone创建新进程时，标志位为CLONE_NEW*将会创建新的Namespace并将子进程放入该Namespace，unshare方法将调用进程放入不同的Namespace中。

2. Mount Namespace

```
struct nsproxy {
    atomic_t count;
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns;
    struct net          *net_ns;
};
```

■ Linux中有一个很核心的思想，那就是一切皆文件。在该思想下，Linux通过挂载对不同设备中的文件进行管理。在Linux中，每一个空目录/文件都可以成为挂载点并设置相应的属性。在Mount Namespace下，处在当前Namespace中的进程只对当前Namespace中的挂载点可见，通过/proc/[pid]/mounts、/proc/[pid]/mountinfo和/proc/[pid]/mountstats 提供不同维度的数据。每个task（在Linux中，不论是进程还是线程，在内核的视角都是一个task）都会指向一个Namesapce（存放在task→nsproxy中）。

但Mount Namespace的引入也带来了新的问题，由于Mount Namespace中的隔离性，当用户需要挂载一个新的磁盘使所有Namespace可见时，就需要在所有的Namespace中都进行一次挂载，很麻烦，于是2.6.15中引入了共享子树（Shared Subtrees）。通过在不同挂载点设置不同的属性，使挂载事件在不同的维度（peer group）进行传播。目前支持以下四种传播类型，其中MS_SHARED和MS_SLAVE比较常见。1. MS_SHARED、2. MS_PRIVATE、3. MS_SLAVE、4. MS_UNBINDABLE。

在MS_SLAVE传播属性的挂载点下，父挂载点（Master）的传播事件可以接收，但子挂载点下（Slave）的挂载事件不再传播，容器的Rootfs挂载即为该种类型，也就是说在容器中挂载的挂载动作是不影响宿主机的，保证了容器隔离。

```
759 func prepareRoot(config *configs.Config) error {
760     flag := unix.MS_SLAVE | unix.MS_REC
761     if config.RootPropagation != 0 {
762         flag = config.RootPropagation
763     }
764     if err := unix.Mount("", "/", "", uintptr(flag), ""); err != nil {
765         return err
766     }
767
768     // Make parent mount private to make sure following bind mount does
769     // not propagate in other namespaces. Also it will help with kernel
770     // check pass in pivot_root. (IS_SHARED(new_mnt->mnt_parent))
771     if err := rootfsParentMountPrivate(config.Rootfs); err != nil {
772         return err
773     }
774
775     return unix.Mount(config.Rootfs, config.Rootfs, "bind", unix.MS_BIND|unix.MS_REC, "")
776 }
```

图13-MS_SLAVE传播类型

4.2.2. 漏洞分析

1. RunC漏洞挂载逻辑分析



图14-整体流程

- checkout到修复commit(0ca91f44f1664-da834bc61115a849b56d22f595f)的上一个版本commitc (01a56034f5ab0c1aa314377a499fe60a9c26b36)。整体流程如左图14

RunC通过命令runc create + runc start 或runc run启动一个容器，runc create主要分为两部分，一部分是准备容器进程的启动参数，与真正实施容器runc init进程进行交互，保证容器初始化顺利进行；另外一部分是执行克隆出的runc init进程，加入各种namespace并初始化容器进程的执行环境。本文以第二部分为切入点进行分析，从libcontainer/standard_init_linux.go的linuxStandardInit.Init()开始，在其中调用prepareRootfs，准备初始化rootfs并进行挂载。

```
48 func (l *linuxStandardInit) Init() error {
49     runtime.LockOSThread()
50     defer runtime.UnlockOSThread()
51     if l.config.Config.NoNewKeyring {...}
52     // 准备网络
53     if err := setupNetwork(l.config); err != nil { err }
54     if err := setupRoute(l.config.Config); err != nil { err }
55
56     // initialises the labeling system
57     selinux.GetEnabled()
58     if err := prepareRootfs(l.pipe, l.config); err != nil {
59         return err
60     }
61     // Set up the console. This has to be done *before* we finalize the rootfs,
62     // but *after* we've given the user the chance to set up all of the mounts
63     // they wanted.
64     if l.config.CreateConsole {
65         if err := setupConsole(l.consoleSocket, l.config, true); err != nil { err }
66         if err := system.Setctty(); err != nil { errors.Wrap(err, "setctty") }
67     }
68
69     // Finish the rootfs setup.
70     if l.config.Config.Namespaces.Contains(configs.NEWNS) {
71         if err := finalizeRootfs(l.config.Config); err != nil { err }
72     }
73 }
```

- 在prepareRootfs 中，调用 prepareRoot 设置初始挂载点，并设置挂载标志位为 unix-MS_SLAVE | unix.MS_REC，其后使用 mountToRootfs 对container.json中配置的挂载进行操作。
- prepareRoot 中设置容器根目录挂载标志位为unix.MS_SLAVE | unix.MS_REC，容器在初始的时候会通过镜像中的容器标准包（bundle）挂载根文件系统（BaseFS），在这里runC默认将挂载点（Propagation Type）设置为slave。由于当前已经处于容器的mount namespace中，所以当前为容器根路径。rootfsParentMountPrivate 函数确保上一层的挂载点是PRIVATE，应该是出于防止逃逸的考虑。

图15-调用prepareRootfs

```
759 func prepareRoot(config *configs.Config) error {
760     flag := unix.MS_SLAVE | unix.MS_REC
761     if config.RootPropagation != 0 {
762         flag = config.RootPropagation
763     }
764     if err := unix.Mount("", "/", "", uintptr(flag), ""); err != nil {
765         return err
766     }
767
768     // Make parent mount private to make sure following bind mount does
769     // not propagate in other namespaces. Also it will help with kernel
770     // check pass in pivot_root. (IS_SHARED(new_mnt->mnt_parent))
771     if err := rootfsParentMountPrivate(config.Rootfs); err != nil {
772         return err
773     }
774
775     return unix.Mount(config.Rootfs, config.Rootfs, "bind", unix.MS_BIND|unix.MS_REC, "")
776 }
```

图16-设置挂载标志位

```
759 func prepareRoot(config *configs.Config) error {
760     flag := unix.MS_SLAVE | unix.MS_REC
761     if config.RootPropagation != 0 {
762         flag = config.RootPropagation
763     }
764     if err := unix.Mount("", "/", "", uintptr(flag), ""); err != nil {
765         return err
766     }
767
768     // Make parent mount private to make sure following bind mount does
769     // not propagate in other namespaces. Also it will help with kernel
770     // check pass in pivot_root. (IS_SHARED(new_mnt->mnt_parent))
771     if err := rootfsParentMountPrivate(config.Rootfs); err != nil {
772         return err
773     }
774
775     return unix.Mount(config.Rootfs, config.Rootfs, "bind", unix.MS_BIND|unix.MS_REC, "")
776 }
```

图17-设置容器根目录挂载标志位


```
331 func mountToRootfs(m *configs.Mount, c *mountConfig) error {
332     rootfs := c.root
333     mountLabel := c.label
334     dest := m.Destination
335     if !strings.HasPrefix(dest, rootfs) {
336         dest = filepath.Join(rootfs, dest)
337     }
338
339     switch m.Device {
340     case "proc", "sysfs":...
341     case "mqueue":...
342     case "tmpfs":...
343     case "bind":...
344     case "cgroup":...
345     default:
346         // ensure that the destination of the mount is resolved of symlinks at mount time because
347         // any previous mounts can invalidate the next mount's destination.
348         // this can happen when a user specifies mounts within other mounts to cause breakouts or
349
350     other
351         // evil stuff to try to escape the container's rootfs.
352         var err error
353         if dest, err = secureJoin.SecureJoin(rootfs, m.Destination); err != nil {
354             return err
355         }
356         if err := checkProcMount(rootfs, dest, m.Source); err != nil {
357             return err
358         }
359         // update the mount with the correct dest after symlinks are resolved.
360         m.Destination = dest
361         if err := os.MkdirAll(dest, perm:0755); err != nil {
362             return err
363         }
364         return mountPropagate(m, rootfs, mountLabel)
365     }
366     return nil
367 }
```

图18–各处理逻辑

```
366 case "tmpfs":
367     copyUp := m.Extensions&configs.EXT_COPYUP == configs.EXT_COPYUP
368     tmpDir := ""
369     // dest might be an absolute symlink, so it needs
370     // to be resolved under rootfs.
371     dest, err := secureJoin.SecureJoin(rootfs, m.Destination)
372     if err != nil & err != {
373         m.Destination = dest
374         stat, err := os.Stat(dest)
375         if err != nil {
376             if err := os.MkdirAll(dest, perm:0755); err != nil {
377                 return err
378             }
379         }
380     }
381     if copyUp {
382         tmpDir, err := prepareTmp(tmpDir: "/tmp")
383         if err != nil {
384             return newSystemErrorWithCause(err, "tmpcopyup: failed to setup tmpdir")
385         }
386         defer cleanupTmp(tmpDir)
387         tmpDir, err = ioutil.TempDir(tmpDir, pattern: "runc*tmpdir")
388         if err != nil {
389             return newSystemErrorWithCause(err, "tmpcopyup: failed to create tmpdir")
390         }
391         defer os.RemoveAll(tmpDir)
392         m.Destination = tmpDir
393     }
394     if err := mountPropagate(m, rootfs, mountLabel); err != nil {
395         return err
396     }
397     if copyUp {
398         if err := fileutils.CopyDirectory(dest, tmpDir); err != nil {
399             errMsg := fmt.Errorf("tmpcopyup: failed to copy %s to %s: %v", dest, tmpDir, err)
400             if err1 := unix.Unmount(tmpDir, unix.MNT_DETACH); err1 != nil { ... }
401             return errMsg
402         }
403         if err := unix.Mount(tmpDir, dest, "", unix.MS_MOVE, ""); err != nil {
404             errMsg := fmt.Errorf("tmpcopyup: failed to move mount %s to %s: %v", dest, tmpDir, err)
405             if err1 := unix.Unmount(tmpDir, unix.MNT_DETACH); err1 != nil { ... }
406             return errMsg
407         }
408     }
409 }
410 if stat != nil {
411     if err := os.Chmod(dest, stat.Mode()); err != nil {
412         return err
413     }
414 }
415 // Initially mounted rw in mountPropagate, remount to ro if flag set.
416 if m.Flags&unix.MS_RDONLY != 0 {
417     if err := remount(m, rootfs); err != nil {
418         return err
419     }
420 }
421 return nil
422 }
```

图19–tmpfs处理逻辑

- 在mountToRootfs 中，针对不同的设备类型存在不同的处理逻辑。

- 在tmpfs的处理逻辑中，configs.EXT_–COPYUP默认为1。

2. 结论

RunC为了防止在路径组合中的路径穿越漏洞，引入了filepath–securejoin作为符号链接过滤函数，但r在挂载时并未校验挂载的实际目的路径，从而导致存在TOCTOU条件竞争漏洞。从securejoin的Readme中也可看出这一点。

NOTE: This code is *only* safe if you are not at risk of other processes modifying path components after you've used SecureJoin . If it is possible for a malicious process to modify path components of the resolved path, then you will be vulnerable to some fairly trivial TOCTOU race conditions. **There are some Linux kernel patches I'm working on which might allow for a better solution.**

In addition, with a slightly modified API it might be possible to use 0_PATH and verify that the opened path is actually the resolved one -- but I have not done that yet. I might add it in the future as a helper function to help users verify the path (we can't just return /proc/self/fd/<foo> because that doesn't always work transparently for all users).

图20–securejoin的Readme

之所以能够成功逃逸的另一原因在于tmpfs中为了实现copy–up功能，使用MS_MOVE作为挂载标志，根据runC作者的描述只有在tmpfs情况才能够逃逸。

4.2.3. 补丁分析

在补丁中，可以看出在tmpfs的挂载逻辑中，增加了doTmpfsCopyUp 函数。

```
return label.SetFileLabel(dest, mountLabel)
case "tmpfs":
    copyUp := m.Extensions&configs.EXT_COPYUP == configs.EXT_COPYUP
    tmpDir := ""
    // dest might be an absolute symlink, so it needs
    // to be resolved under rootfs.
    dest, err := secureJoin.SecureJoin(rootfs, m.Destination)
    if err != nil {
        return err
    }
    m.Destination = dest
    stat, err := os.Stat(dest)
    if err != nil {
        if err := os.MkdirAll(dest, 0755); err != nil {
            return err
        }
    }
    if copyUp {
        tmpDir, err := prepareTmp("/tmp")
        if err != nil {
            return newSystemErrorWithCause(err, "tmpcopyup: failed to setup tmpdir")
        }
        defer cleanupTmp(tmpDir)
        tmpDir, err = ioutil.TempDir(tmpDir, "runc*tmpdir")
        if err != nil {
            return newSystemErrorWithCause(err, "tmpcopyup: failed to create tmpdir")
        }
        defer os.RemoveAll(tmpDir)
        m.Destination = tmpDir
    }
    if err := mountPropagate(m, rootfs, mountLabel); err != nil {
        return err
    }
    if copyUp {
        if err := fileutils.CopyDirectory(dest, tmpDir); err != nil {
            errMsg := fmt.Errorf("tmpcopyup: failed to copy %s to %s: %v", dest, tmpDir, err)
            if err1 := unix.Unmount(tmpDir, unix.MNT_DETACH); err1 != nil {
                return newSystemErrorWithCause(err1, "tmpcopyup: %v: failed to unmount", errMsg)
            }
            return errMsg
        }
        if err := unix.Mount(tmpDir, dest, "", unix.MS_MOVE, ""); err != nil {
            errMsg := fmt.Errorf("tmpcopyup: failed to move mount %s to %s: %v", tmpDir, dest, err)
            if err1 := unix.Unmount(tmpDir, unix.MNT_DETACH); err1 != nil {
                return newSystemErrorWithCause(err1, "tmpcopyup: %v: failed to unmount", errMsg)
            }
            return errMsg
        }
    }
    if stat != nil {
        if err := os.Chmod(dest, stat.Mode()); err != nil {
            return err
        }
    }
}

// bind mount won't change mount options, we need remount to make mount options effective.
// first check that we have non-default options required before attempting a remount
if m.Flags&unix.MS_REC|unix.MS_REMOUNT|unix.MS_BIND != 0 {
    // only remount if unique mount options are set
    if err := remount(m, rootfs); err != nil {
        return err
    }
}
```

图21–补丁新增

4.2.5. 总结

Linux在引入symlink的时候并不存在安全风险，但随着时代的变迁（容器的引入），symlink确实在一定程度上确实容易造成容器逃逸的发生。Linux在尝试在不同的角度去解决这个问题，但目前还没有很完全的能够解决此风险。这里不禁让人想引用tk的一句话：“安全意识要有时代背景”

作者认为伴随容器场景愈发复杂，安全研究的逐渐深入，非Linux内核漏洞导致的容器逃逸长期来看还会有一个增长的趋势。

4.3 浅谈云上攻防一元数据服务带来的安全挑战

4.3.1. 前言

在针对云上业务的的攻击事件中，很多攻击者将攻击脆弱的元数据服务作为攻击流程中重要的一个环节并最终造成了严重的危害。

以2019年的美国第一资本投资国际集团（Capital One）信息泄露事件举例，根据《A Case Study of the Capital One Data Breach》报告指出，攻击者利用Capital One部署在AWS云上实例中的SSRF漏洞向元数据服务发送请求并获取角色的临时凭证，在获取角色临时凭据后将该角色权限下的S3存储桶中的数据复制到攻击者的本地机器上，最终导致这一严重数据泄露事件的产生，这一事件影响了北美超过1亿人。Capital One 的股价在宣布数据泄露后收盘下跌 5.9%，在接下来的两周内总共下跌了 15%。

Capital One信息泄露事件攻击原理图，可参见图15：

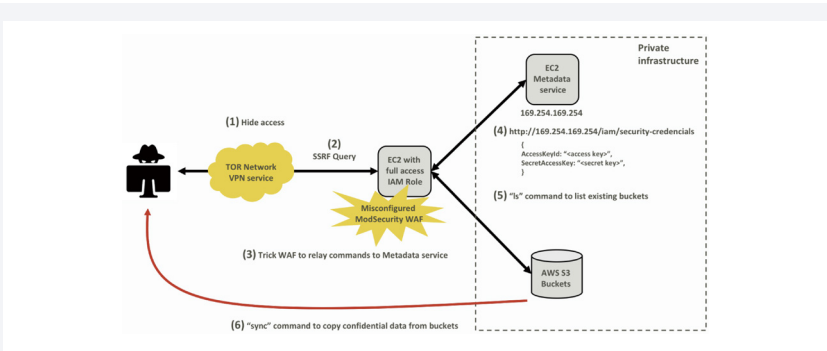


图24-攻击原理图

```
334 func doTmpfsCopyUp(m *configs.Mount, rootfs, mountLabel string) (Err error) {
335     // Set up a scratch dir for the tmpfs on the host.
336     tmpDir, err := prepareTmp(tmpDir := "/tmp")
337     if err != nil {
338         return newSystemErrorWithCause(err, cause: "tmpcopyup: failed to setup tmpdir")
339     }
340     defer cleanupTmp(tmpDir)
341     tmpDir, err := ioutil.TempDir(tmpDir, pattern: "runctmpdir")
342     if err != nil {
343         return newSystemErrorWithCause(err, cause: "tmpcopyup: failed to create tmpdir")
344     }
345     defer os.RemoveAll(tmpDir)
346
347     // Configure the *host* tmpdir as if it's the container mount. We change
348     // m.Destination since we are going to mount *on the host*.
349     oldDest := m.Destination
350     m.Destination = tmpDir
351     err = mountPropagate(m, rootfs: "/", mountLabel)
352     m.Destination = oldDest
353     if err != nil {
354         return err
355     }
356     defer func() {
357         if Err != nil {
358             if err := unix.Unmount(tmpDir, unix.MNT_DETACH); err != nil {
359                 logrus.Warnf("tmpcopyup: failed to unmount tmpdir on error: %s", err)
360             }
361         }
362     }()
363
364     return utils.WithProcfd(rootfs, m.Destination, func(procfd string) (Err error) {
365         // Copy the container data to the host tmpdir. We append "/" to force
366         // CopyDirectory to resolve the symlink rather than trying to copy the
367         // symlink itself.
368         if err := fileutils.CopyDirectory(procfd+"/", tmpDir); err != nil {
369             return fmt.Errorf("tmpcopyup: failed to copy %s to %s: %s", m.Destination, procfd, err)
370         }
371         // Now move the mount into the container.
372         if err := unix.Mount(tmpDir, procfd, "", unix.MS_MOVE, ""); err != nil {
373             return fmt.Errorf("tmpcopyup: failed to move mount %s to %s: %s", tmpDir, procfd, err)
374         }
375         return ErrNil
376     })
377 }
```

图22-处理内容

```
112 // WithProcfd runs the passed closure with a procfd path (/proc/self/fd/...)
113 // corresponding to the unsafePath resolved within the root. Before passing the
114 // fd, this path is verified to have been inside the root -- so operating on it
115 // through the passed fdpath should be safe. Do not access this path through
116 // the original path strings, and do not attempt to use the pathname outside of
117 // the passed closure (the file handle will be freed once the closure returns).
118 func WithProcfd(root, unsafePath string, fn func(procfd string) error) error {
119     // Remove the root then forcefully resolve inside the root.
120     unsafePath = stripRoot(root, unsafePath)
121     path, err := securejoin.SecureJoin(root, unsafePath)
122     if err != nil {
123         return fmt.Errorf("resolving path inside rootfs failed: %s", err)
124     }
125
126     // Open the target path.
127     fh, err := os.OpenFile(path, unix.O_PATH|unix.O_CLOEXEC, perm: 0)
128     if err != nil {
129         return fmt.Errorf("open o_path procfd: %s", err)
130     }
131     defer fh.Close()
132
133     // Double-check the path is the one we expected.
134     procfd := "/proc/self/fd/" + strconv.Itoa(int(fh.Fd()))
135     if realpath, err := os.Readlink(procfd); err != nil {
136         return fmt.Errorf("procfd verification failed: %s", err)
137     } else if realpath != path {
138         return fmt.Errorf("possibly malicious path detected -- refusing to operate on %s", realpath)
139     }
140
141     // Run the closure.
142     return fn(procfd)
143 }
```

图23-配置文件

■ 在其中使用WithProcfd 函数防止TOCTOU漏洞的发生，所有的 securejoin.SecureJoin 移入WithProcfd进行统一处理。

■ WithProcfd 中使用/proc/self/fd/ ，确保打开的文件是securejoin.SecureJoin 后的文件。

4.2.4. POC分析

漏洞作者给出的POC中给出了一个很精妙的构造，利用了这个看似很难利用的条件竞争漏洞。首先，创建两个公共tmpfs的挂载，名称为test1、test2，在容器A中，将test1挂载到/test1路径，test2挂载到/test2路径，同时将/test2/test2指向/。在容器B中，将test1挂载到/test1路径，test2挂载到/test1/mntx路径和/test1/zzz路径。

在容器A启动后，将/test1/mnt-tmpx指向rootfs路径，且交换mnt和mnt-tmpx，且rootfs/test2/test2指向/（K8S中，同一个pod下的rootfs在一个路径，形如/var/lib/kubelet/pods/\$MY_POD_UID/volumes/kuber-netes.io~empty-dir）。

所以当条件竞争挂载的时候，即容器B启动时，挂载test2，mount('/', 'rootfs/test1/zzz')，同时MS_MOVE标志位将原有该挂载点的subtree移至新挂载点下，造成逃逸发生。

在介绍元数据服务带来的安全挑战之前，我们先来简单介绍一下元数据服务以及角色的概念。

4.2.4. POC分析

1. 元数据服务

元数据即表示实例的相关数据，可以用来配置或管理正在运行的实例。用户可以通过元数据服务在运行中的实例内查看实例的元数据。以AWS举例，可以在实例内部访问如下地址来查看所有类别的实例元数据：
http://169.254.169.254/latest/meta-data/

169.254.169.254属于链路本地地址（Link-local address），链路本地地址又称连结本地位址，是计算机网络中一类特殊的地址，它仅供于在网段，或广播域中的主机相互通信使用。这类主机通常不需要外部互联网服务，仅有主机间相互通讯的需求。IPv4链路本地地址定义在169.254.0.0/16地址块。

而在具体的技术实现上，云厂商将元数据服务运行在Hypervisor（虚拟机管理程序）上。当实例向元数据服务发起请求时，该请求不会通过网络传输，也永远不会离开这一台计算机。基于这个原理，元数据服务只能从实例内部访问。

可以PING云厂商所提供的元数据服务域名，以查看其IP地址

```
PING metadata.169.254.0.23.com (169.254.0.23) 56(84) bytes of data.  
64 bytes from 169.254.0.23: icmp_seq=1 ttl=64 time=0.304 ms  
64 bytes from 169.254.0.23: icmp_seq=2 ttl=64 time=0.353 ms  
64 bytes from 169.254.0.23: icmp_seq=3 ttl=64 time=0.337 ms
```

图25--元数据服务域名

从上图可见，元数据服务属于链路本地地址。从设计上来看，元数据服务看起来很安全，那为什么说元数据服务脆弱呢？

由于元数据服务部署在链路本地地址上，云厂商并没有进一步设置安全措施来检测或阻止由实例内部发出的恶意的对元数据服务的未授权访问。攻击者可以通过实例上应用的SSRF漏洞对实例的元数据服务进行访问。

因此，如果实例中应用中存在SSRF漏洞，那么元数据服务将会完全暴露在攻击者面前。在实例元数据服务提供的众多数据中，有一项数据特别受到攻击者的青睐，那就是角色的临时访问凭据。这将是攻击者由SSRF漏洞到获取实例控制权限的桥梁。

2. 访问管理角色

既然攻击涉及到访问管理角色的临时凭据，我们首先看下访问管理角色是什么：访问管理的角色是拥有一组权限的虚拟身份，用于对角色载体授予云中服务、操作和资源的访问权限。用户可以将角色关联到云服务器实例。为实例绑定角色后，将具备以下功能及优势：

- 可使用 STS 临时密钥访问云上其他服务
- 可为不同的实例赋予包含不同授权策略的角色，使实例对不同的云资源具有不同的访问权限，实现更精细粒度的权限控制
- 无需自行在实例中保存 SecretKey，通过修改角色的授权即可变更权限，快捷地维护实例所拥有的访问权限

具体的操作流程如下：



图26--操作流程

在将角色成功绑定实例后，用户可以在实例上访问元数据服务来查询此角色的临时凭据，并使用获得的临时凭据操作该角色权限下的云服务API接口。

4.3.3. 针对元数据服务的攻击

接下来我们将介绍下针对元数据服务的一些常见的攻击模式。攻击者可以首先通过目标实例上的SSRF漏洞获取与实例绑定的角色名称（rolename）。攻击者可以构造访问元数据接口的payload，并通过存在SSRF漏洞的参数传递：

http://x.x.x.x/?url=http://169.254.169.254/latest/meta-data/iam/info在获取到角色名称后，攻击者可以继续通过SSRF漏洞获取角色的临时凭证：

http://x.x.x.x/?url=http://169.254.169.254/latest/metadata/iam/security-credentials/<rolename>获取角色临时凭据的案例可参见下图18:

```
...latest/meta-data/iam/security-credentials/ruiqianggao  
{"TmpSecretId": "COawtr0EOx2hQRmDetBbdKJ8TC7ZXjGa14672674f93607ad373b920669d34be8N8h6", "TmpSecretKey": "D9n_s8hROiilr7Zn8e3Cv0CvKdNSMmv_cegT_dOncwmlCOvZH8T2ji84yikaYg4T9wFFWSNch_MGvHfe1Jdk0nvSJekO8WaXslikZEOKMI2g_W2NKzPCo2onTulVuFS", "Go_cXe": "Code": "Success"}
```

图27--获取临时凭据

从上图可见，攻击者可以获取角色的TmpSecretID以及TmpSecretKey。在攻击者成功获取角色的临时凭据后，将会检查获取到的角色临时凭据的权限策略。有的时候，可以通过获取到的角色名称，来猜测该角色的权限策略，例如角色名为：TKE_XXX，则这个角色很大可能是拥有操作TKE容器服务的权限。此外，如果获取的临时密钥拥有查询访问管理接口的权限，攻击者可以通过访问“访问管理”API来准确获取的角色权限策略。可以通过如下几种方式判断获取角色的权限策略：

1. 通过使用临时API凭据访问“获取角色绑定的策略列表”API接口，见下图：



图28–“获取角色绑定的策略列表”API接口

从上图可见，攻击者获取到的与实例绑定的角色的临时凭据权限策略是“AdministratorAccess”,这个策略允许管理账户内所有用户及其权限、财务相关的信息、云服务资产。

2. 通过使用临时API凭据访问“获取角色详情”API接口，见下图：



图29–“获取角色详情”API接口

通过查询的返回结果可以见，角色的权限策略为AssumeRole。在弄清楚窃取的凭据所拥有的权限后，攻击者便可以通过凭据的权限制定后续的攻击流程。但在开始后续的攻击阶段之前，攻击者会先判断当前权限是否可以获取目标的数据资源。在所有云资源中，攻击者们往往对目标的数据更加感兴趣。如果攻击者获取的密钥拥有云数据库服务或云存储服务等服务操作权限，攻击者将会尝试窃取目标数据。临时凭据同样也可以帮助攻击者们在目标实例中执行指令并控制实例权限。与通过密钥构造请求这种方式发起攻击相比，攻击者们在实战中更倾向于使用云命令行工具来进行攻击。云服务厂商为用户提供了相应的云命令行工具以管理云服务，例如腾讯云提供的TCCLI工具、AWS的AWS CLI工具。攻击者可以通过在云命令行工具中配置窃取到的API密钥来对云资源进行调用。与构造请求访问云API接口这种方式相比，使用云命令行工具将会给攻击者带来更多便捷。

在使用云命令行工具之前，应先配置API密钥，以AWS CLI工具配置举例，可以将：



图30–配置密钥

攻击者将窃取来的AWS_ACCESS_KEY_ID、AWS_SECRET_ACCESS_KEY、AWS_SESSION_TOKEN配置完成后，可以使用云命令行工具在目标实例上执行命令。在配置好密钥后，攻击者可以尝试使用如下图命令通过AWS CLI在实例中运行bash脚本以获取实例控制权限



图31–获取权限

借助通过元数据服务窃取到的凭据以及AWS CLI所提供的功能,攻击者可以在实例中执行反弹shell命令，由此进入实例。

除此之外，攻击者还可以选择修改userdata，将反弹shell写入userdata中后将实例重启，从而控制实例。

Userdata涉及到云厂商提供了一种功能，这项功能允许用户自定义配置在实例启动时执行的脚本的内容。

通过这一功能，攻击者可以尝试在实例的userdata中写入恶意代码，这些代码将会在实例每次启动时自动执行。

通过这一功能，攻击者可以尝试在实例的userdata中写入恶意代码，这些代码将会在实例每次启动时自动执行。

以AWS举例，攻击者可以将恶意代码写入my_script.txt文件中，然后执行如下指令将my_script.txt文件中内容导入userdata中

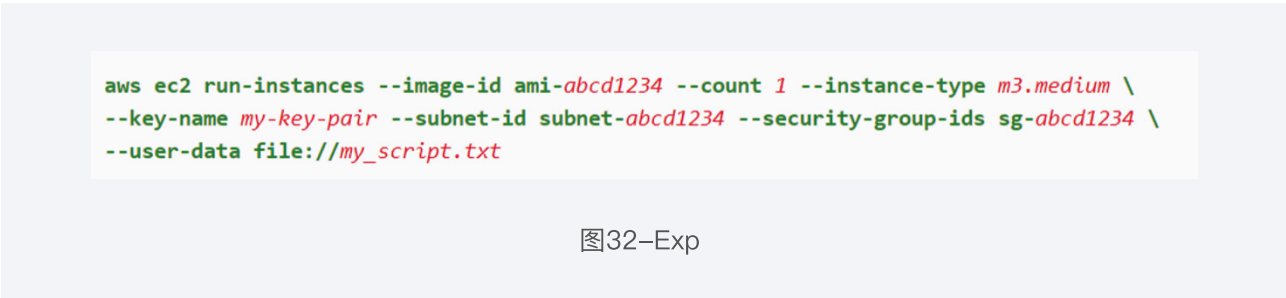


图32–Exp

随后，攻击者通过如下命令重启实例：

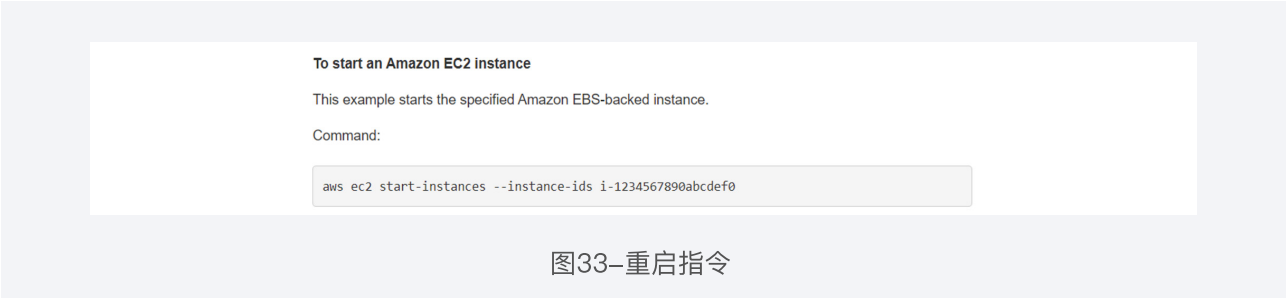


图33–重启指令

当实例重启时，userdata中的恶意代码将会被执行。

攻击者除了可以使用临时凭据获取实例的控制权限，通过元数据服务窃取到的拥有一定权限的角色临时凭据在持久化阶段也发挥着作用。攻击者尝试使用通过元数据服务获取的临时凭据进行持久化操作，确保能够持续拥有访问权限，以防被发现后强行终止攻击行为。

使用临时凭据进行持久化的方式有很多，比如说在上文中所提及的在userdata中写入恶意代码这项攻击技术，也是可以运用在持久化阶段：通过在实例的userdata中写入恶意代码，这些代码将会在实例每次启动时自动执行。这将很好的完成持久化操作而不易被发现。

除此之外，攻击者还可以尝试在账户中创建一个新的用户以进行持久化，以AWS CLI举例，攻击者可以通过 `aws iam create-user --user-name Bob` 为账户新建一个名为Bob的用户



图34–新建用户操作

随后使用 `aws iam create-access-key --user-name Bob` 指令为Bob用户创建凭据

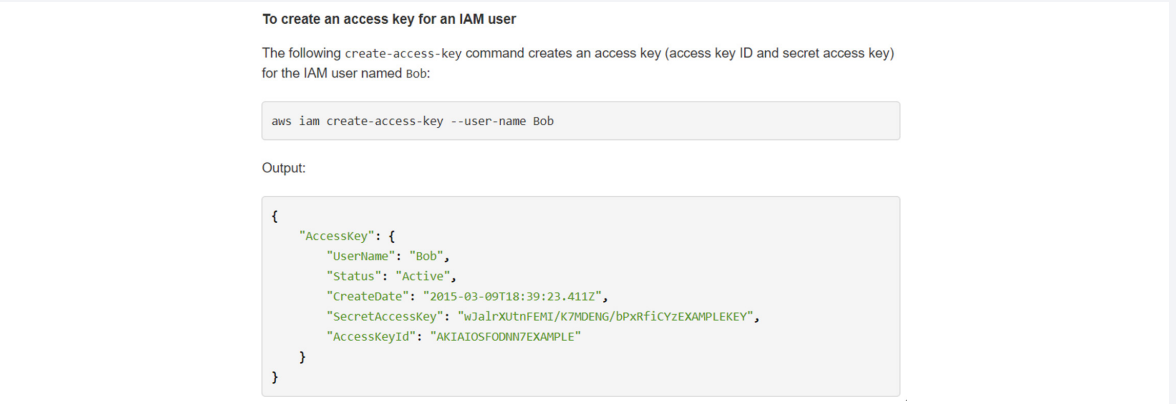


图35–创建凭据

虽然这个方法操作简单且有效，但是账户里突然新增的用户及其容易被察觉，因此并不是一个特别有效的持久化方式。

此外，攻击者还会使用一种常见的持久化手法，那就是给现有的用户分配额外的密钥。以针对AWS的攻击来说，攻击者可以使用aws_pwn这款工具来完成这项攻击，aws_pwn地址如下：
https://github.com/dagrz/aws_pwn

aws_pwn提供了多项技术以共攻击者可以完成针对aw的持久化攻击，关于aws_pwn所提供的持久化功能可见下图：

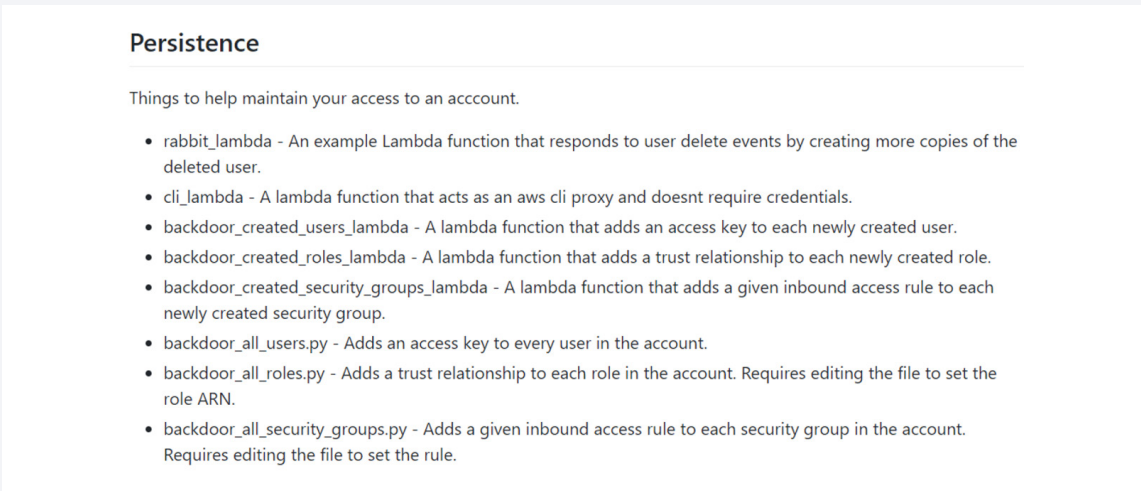


图36–aws_pwn持久化功能

通过元数据服务窃取也可以被攻击者应用于横向移动操作。攻击者可以通过元数据服务窃取角色的临时凭据横向移动到角色对应权限的资源上。除此之外，攻击者会在所控制的实例上寻找配置文件，并通过配置文件中的配置项中获取其他资源的访问方式以及访问凭据。

攻击者在横向移动的过程中，获取到可以操作云数据库或存储服务必要权限的密钥或是登录凭据后，攻击者就可以访问这些服务并尝试将其中的用户数据复制到攻击者的本地机器上。

以AWS CLI为例，攻击者可以通过如下命令将s3存储桶中的内容同步到本地

```
$ aws s3 sync <source> <target> [--options]
```

图37–同步命令

仍然以上文提及的Capital One银行数据泄露事件举例，攻击者使用获取到的角色临时凭据，多次执行“aws s3 ls”命令，获取Capital One 账户的存储桶的完整列表；

接着攻击者使用 sync命令将近 30 GB 的 Capital One用户数据复制到了攻击者本地。

总的来说，元数据服务为云上安全带来了极大的安全挑战，攻击者在通过SSRF等漏洞获取到实例绑定的角色的临时凭据后，将会将其应用于云上攻击的各个阶段。通过破坏用户系统，滥用用户资源、加密用户资源并进行勒索等手段影响用户环境正常使用。

4.3.4. 元数据安全性改进

以AWS为例，AWS为了解决元数据服务在 SSRF 攻击面前暴露出的安全性问题，引入IMDSv2来改善其总体安全情况。

在IMDSv2中，如果用户想访问元数据服务，首先需要在实例内部向IMDSv2发送一个HTTP PUT请求来启动会话，示例如下：

```
curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"
```

图38-访问操作

X-aws-ec2-metadata-token-ttl-seconds用于指定生存时间（TTL）值（以秒为单位），上文中生成的token有效期为6小时（21600秒），在IMDSv2中21600秒是允许的最大TTL值。此请求将会返回一个token，后续访问元数据服务，需要在HTTP header中携带此token，见如下请求

```
curl http://169.254.169.254/latest/meta-data/profile -H "X-aws-ec2-metadata-token: $TOKEN"
```

图39-携带token

完整流程如下：
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-to-ken-ttl-seconds: 21600"
curl http://169.254.169.254/latest/meta-data/profile -H "X-aws-ec2-metadata-token: \$TOKEN"

流程图如下：

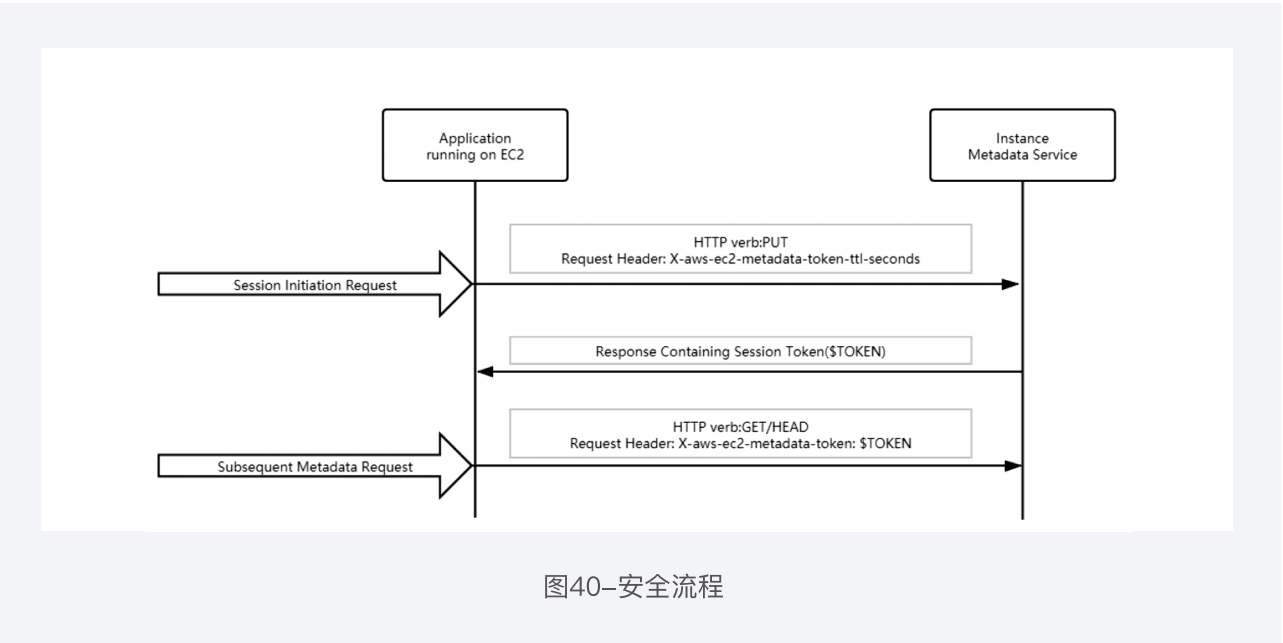


图40-安全流程

可见，在采用IMDSv2时，即使实例中应用存在SSRF漏洞，攻击者也无法轻易的利用SSRF漏洞向元数据服务发出PUT请求来获取token，在没有token的情况下，攻击者并不能访问元数据服务，也就无法获取角色的临时凭据进行后续的攻击行为。

除了使用PUT启动请求这项安全策略之外，IMDSv2还引入了如下两个机制保证元数据服务的安全：

不允许X-Forwarded-For标头：如果攻击者通过反向代理的方式的确可以绕过PUT限制，但是，通过代理传递的请求将包含“X-Forwarded-For”标头。这样的请求被IMDSv2拒绝，并且不发行令牌。

IP数据包TTL设置为“1”：TTL指定数据包被路由器丢弃之前允许通过的最大网段数量，是IP数据包在网络中可以转发的最大跳数(跃点数),将其值设置为1可确保包含机密令牌的HTTP响应不会在实例外部传播。即使攻击者能够绕过所有其他保护措施，这也将确保令牌不会在实例外部传播，并且一旦数据包离开实例，数据包将被丢弃。

值得注意的是，AWS认为现有的实例元数据服务（IMDSv1）是完全安全的，因此将继续支持它。如果不执行任何操作，则IMDSv1和IMDSv2都可用于EC2实例。这就是说，在不主动禁用IMDSv1的情况下，实例仍存在着安全隐患。

4.3.5. 元数据服务更多安全隐患

IMDSv2方案的确可以有效的保护存在SSRF漏洞的实例，使其元数据不被攻击者访问。但是这项技术可以完美的保护元数据、保护租户的云业务安全吗？答案是不能。

设想一下：当攻击者通过其他漏洞（例如RCE漏洞）获取实例的控制权之后，IMDSv2的安全机制将变得形同虚设。攻击者可以在实例上发送PUT请求获取token，随后利用获得的token获取角色临时凭据，最后利用角色临时凭据访问角色绑定的一切云业务，具体流程见下图32：

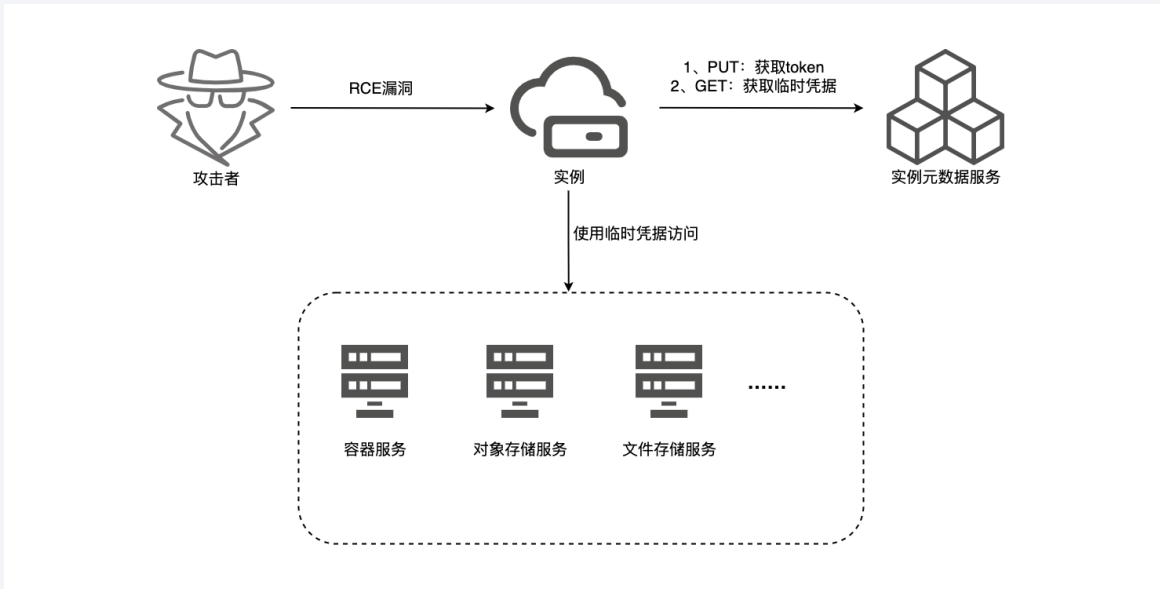


图41-攻击流程

总之，当攻击者通过RCE漏洞获取实例控制权后，可以通过元数据服务获取到的临时凭据进行横向移动。鉴于云厂商产品API功能的强大性，在获取角色临时凭据后，可能造成极其严重的影响

值得注意的是，如果在云平台控制台中执行一些高危行为，平台默认都会需要进行手机验证。但通过使用临时凭据调用发送请求调用API接口，并不需要手机验证码，可以绕过这项安全检测。

4.4 SSRF漏洞对云原生环境带来的新威胁

4.4.1. 前言

在《浅谈云上攻防之一元数据服务带来的安全挑战》一文中，作者生动形象的为我们讲述了元数据服务所面临的一系列安全问题，而其中的问题之一就是通过SSRF去攻击元数据服务。

作者也为我们列举了2019年美国第一资本投资国际集团（CapitalOne）信息泄漏的案例。而我们内部也监测到过类似的事件，测试人员通过SSRF漏洞攻击元数据服务，并将获取到的AK信息存储到日志服务中，然后在日志服务中获取到了AK信息，最终通过获取到的AK信息控制了超过200台服务器。具体攻击路径如图所示。

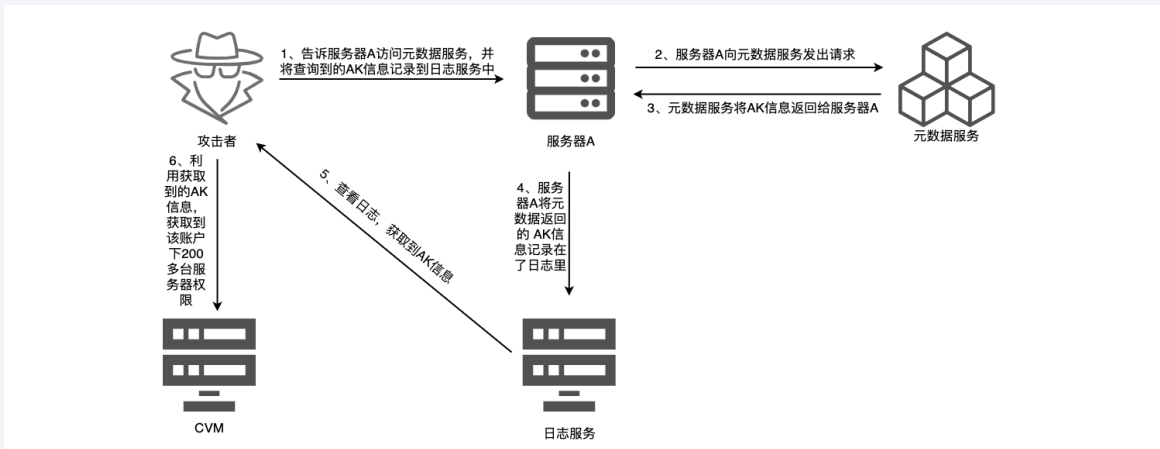


图42-攻击路径

通过上述案例，我们可以看到在云场景中，由于云厂商提供云服务均使用同一套网络边界和鉴权系统，且各云组件默认相互信任。此时一旦存在SSRF漏洞，此类边界将不复存在，攻击者可直接调用云厂商支持环境中的相应接口，因此SSRF漏洞在云环境中更具有危害性。为此本文就SSRF与云环境结合所带来的一些问题以及SSRF常见的一些绕过方法进行了整理，希望通过对这些方法的学习来提高我们在云上对于SSRF的防护能力。

4.4.2. SSRF与云环境结合带来的严重影响

在介绍SSRF漏洞在云场景中的危害之前，这里先为大家简单介绍一下什么是SSRF漏洞。SSRF(Server-Side Request Forgery:服务器端请求伪造)是一种由攻击者构造形成由服务端发起请求的一个安全漏洞。SSRF形成的原因大都是由于服务端提供了对外发起请求的功能且没有对目标地址做过滤与限制。SSRF根据是否回显请求内容分为回显型SSRF和非回显型SSRF。如图所示，当服务器A存在SSRF漏洞时，攻击者就可以通过服务器A去访问内网的服务器B，从而向B服务器发起攻击。

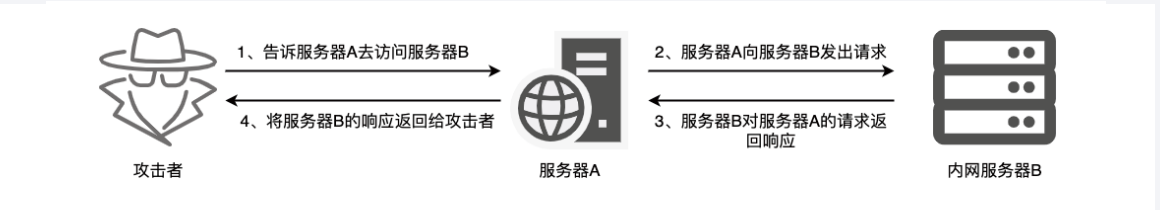


图43-SSRF原理

在传统环境中，SSRF漏洞的主要作用是帮助攻击者突破网络边界，从而可以使攻击者攻击那些外网无法访问的内部系统。而这些内部系统往往都容易成为企业安全建设的盲区。从而导致企业内网被攻破的概率增加。在传统环境中，SSRF漏洞的危害基本可以分为以下四种：

- 获取敏感信息：攻击者通过SSRF可以尝试获取一些存在敏感信息的系统文件或者网页。
- 探测内网：攻击者也可以通过SSRF去对内网的主机和端口进行探测，获取内网主机的IP存活和端口开放信息，从而去判断内网都开有哪些服务。
- 攻击内网应用程序：根据获取到的内网服务的信息，攻击者就可以有针对性的对内网的web应用，或者其他应用程序进行攻击，如weblogic, redis, tomcat等等，从而获取内网机器的权限，为后续的攻击打开突破口。
- DOS攻击：如果有需要，攻击者也可以利用SSRF企业服务进行DOS攻击。

在云环境中，SSRF漏洞除了传统的攻击内网等攻击方式外，也增加了一些云上特有的攻击方式，这些攻击方式一旦被攻击者利用成功，往往都会对云上资产造成严重的危害。下面就为大家一一介绍一下这些攻击方式：

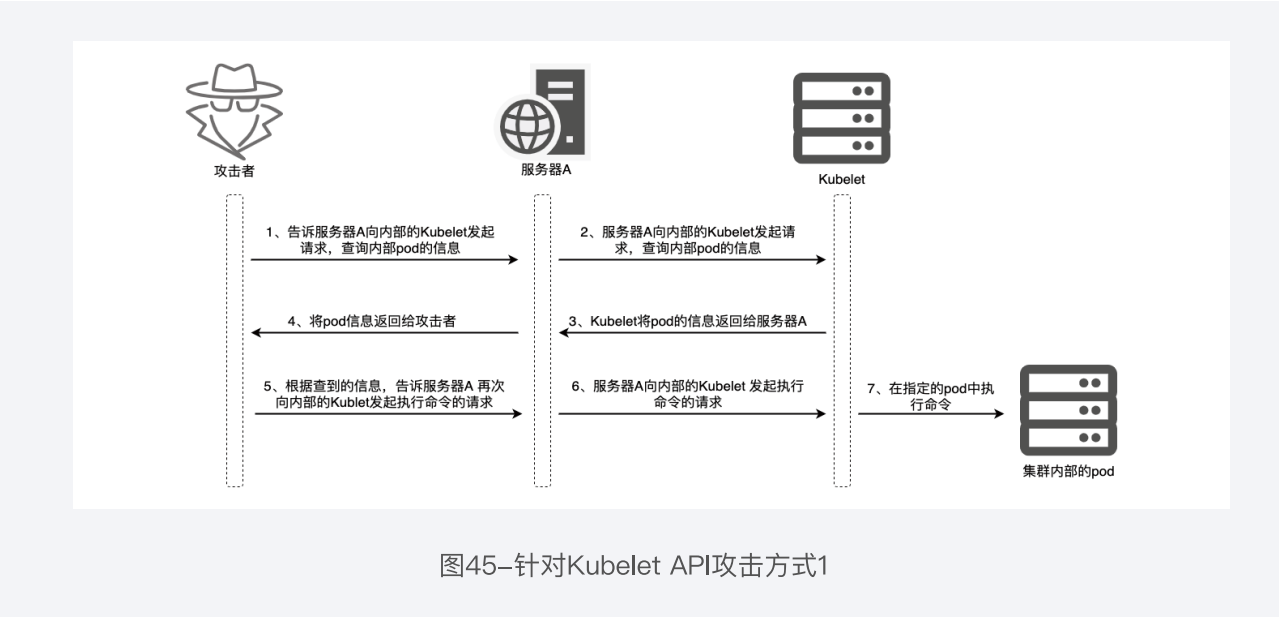
1. 攻击元数据服务

在云环境中，元数据即表示实例的相关数据，可以用来配置或管理正在运行中的实例。攻击通过SSRF去访问元数据中存储的临时密钥或者用于自启动实例的启动脚本，这些脚本可能会包含AK、密码、源码等等，然后根据从元数据服务获取的信息，攻击者可尝试获取到受害者账户下COS、CVM、集群等服务的权限。具体攻击方式如下图所示：

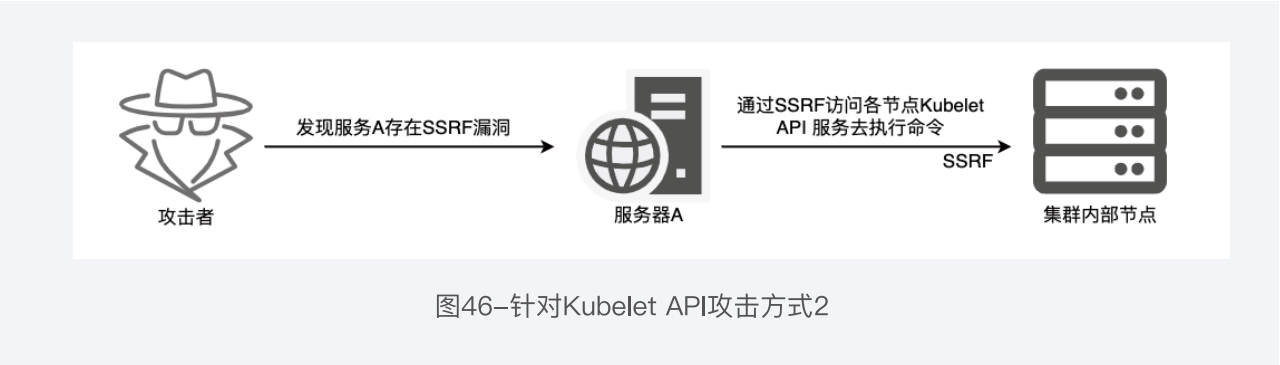


2. 攻击Kubelet API

在云环境中，可通过Kubelet API查询集群pod和node的信息，也可通过其执行命令。为了安全考虑，此服务一般不对外开放。但是，攻击者可以通过SSRF去访问Kubelet API，获取信息和执行命令。具体攻击方式如下图所示：



近期我们也监测到一个类似的案例，如下图所示，测试人员通过发现的某个SSRF漏洞，通过访问集群中的Kubelet API来执行命令，从而获取进群内容器的权限。



3. 攻击Docker Remote

API:Docker Remote API是一个取代远程命令行界面（rccli）的REST API，默认开放端口为2375。此API如果存在未授权访问，则攻击者可以利用其执行docker命令，获取敏感信息，并获取服务器root权限。因此为了安全考虑，一般不会在外网开放，此时我们就可以通过SSRF去尝试攻击那些不对外开放的Docker Remote API。其过程与攻击Kubelet API类似。

4. 越权攻击云平台内其他组件或服务

由于云上各组件相互信任，当云平台内某个组件或服务存在SSRF漏洞时，就可通过此漏洞越权攻击其他组件或者服务。如下图所示，用户正常请求服务A时，云API层会对请求进行校验，其中包括身份、权限等。如果服务A存在SSRF漏洞，则可构造请求使服务A访问服务B，因为服务A与服务B互相信任，所以服务B未校验服务A的请求，从而越权操作服务B的资源。

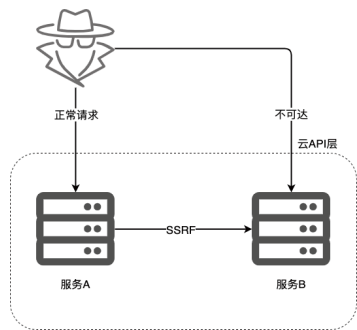


图47-越权操作

由于云场景中业务繁多，因此除了传统的一些可能出现SSRF的场景之外，也出现了一些新的容易出现SSRF的场景：

- 代理功能：如开发、运维人员因为方便工作所搭建的web代理等等，此类功能如果没有设置好身份认证，就会导致SSRF漏洞出现，因为利用方式简单，且存在回显，故此类问题一旦出现，造成的危害往往都会比较严重。
- 各种对外发起网络请求或者需要远程下载资源的功能：此类功能如果没能做好安全检测，就会导致SSRF。如weblogic的SSRF漏洞。
- 文件上传功能：在某些场景中，开发人员未对上传类型进行限制，导致攻击者可以修改type=url，将type=file改为type=url，然后将上传内容改为url，测试可否上传成功，如果可以上传成功，则可能存在SSRF。
- 数据库内置功能：如mongodb的copyDatabase函数，在进行数据备份时，输入的IP进行限制，则可能存在SSRF漏洞。
- 未公开的api：这类api有时也会有对外发起网络请求或者需要远程下载资源的功能，并且因此api未公开，所以很有可能会成为安全防护的盲区。

- 其他：处理图片文件、处理音视频文件、html解析、PDF解析等，这些功能如果防护不到位，也可能存在SSRF漏洞，如PDFReacter等等。
- 对象存储功能：对象存储功能是云上特有的文件存储系统，在对象存储功能中，获取存储桶时，如果此时未做302校验，则可结合cos回源绕过，尝试是否存在SSRF

4.4.3. 自动化收敛云上SSRF漏洞

云环境与SSRF结合虽然带了许多问题，但是也为我们带来了全新的检测SSRF漏洞的可能，我们可以利用云上的一些特性，去尝试寻找一些相比于常规环境下更快速、更高效的SSRF自动化收敛方案。在这样的背景下，我们开创性的提出了一种基于云API的实时监控来检测SSRF漏洞的方法，并实际应用于日常SSRF检测中，这种方式可以基本覆盖接入了云API的云产品，针对存在SSRF漏洞的接口进行精准测试，大大提高了SSRF漏洞的检出率，保障了产品侧以及用户侧的安全。

4.4.4. 写在最后

近些年来，随着云计算产业的蓬勃发展，越来越多的企业选择业务上云，因此云上安全也显得愈发重要，一方面，SSRF漏洞与云结合产生的重大影响，如SSRF与元数据、SSRF与kubelet等等。另一方面，随着云计算技术与SSRF攻击技术的不断发展，SSRF与云也会不断产生新的化学反应，因此需要我们去更加重视这个问题，不断学习新的知识，以攻促防，为云上安全保驾护航。

4.5 国内首个对象存储ATT&CK攻防矩阵

对象存储是云厂商提供了一种用来存储海量文件的分布式存储服务，可用于大规模存储非结构化数据。因为其具有高扩展性、低成本、可靠安全等优点，所以成为许多IT产业向云原生的开发和部署模式转变过程中不可或缺的一部分。

随着云上业务的蓬勃发展，作为云原生的一项重要能力，对象存储服务同样也面临着一系列的安全挑战。纵观近些年的云安全漏洞，与对象存储服务相关的数据泄露事件比比皆是，以2017美国国防部承包商数据泄露为例：

“Booz Allen Hamilton公司（提供情报与防御顾问服务）在使用亚马逊S3服务器存储政府的敏感数据时，由于使用了错误的配置，从而导致了政府保密信息可被公开访问。经安全研究人员发现，公开访问的S3存储桶中包含47个文件和文件夹，其中三个文件可供下载，内部包含了大量“绝密”(TOP SECRET)以及“外籍禁阅”(NOFORN)文件。”

因此，全方面的了解关于对象存储服务的攻击手段以及途径，这将有效的帮助云平台以及云租户在面对这些风险时精准的识别风险并采取相应的防护措施，保护对象存储服务以及其中存储的数据安全。

4.5.1. 对象存储服务攻防矩阵概览

腾讯安全云鼎实验室在MITRE ATT&CK®框架研究的基础上，针对对象存储服务所面临的威胁以及攻击技术进行整理，推出对象存储服务攻击矩阵，用以展现针对对象存储服务的攻击手段以及关键技术，如图所示：



4.5.2. 初始访问

1. 云平台主API密钥泄露

云平台主API 密钥重要性等同于用户的登录密码，其代表了账号所有者的身份以及对应的权限。

API 密钥由SecretId和SecretKey组成，用户可以通过API密钥来访问云平台API进而管理账号下的资源。在一些攻击场景中，由于开发者不安全的开发以及配置，或者一些针对设备的入侵事件，导致云平台主API 密钥泄露，攻击者可以通过窃取到的云平台主API 密钥，冒用账号所有者的身份入侵云平台，非法操作对象存储服务并篡改、窃取其中的数据。

2. 对象存储SDK泄露

云平台所提供的对象存储服务，除了拥有多种 API 接口外，还提供了丰富多样的SDK供开发者使用。

在SDK初始化阶段，开发者需要在SDK中配置存储桶名称、路径、地域等基本信息，并且需要配置云平台的永久密钥或临时密钥，这些信息将会被编写在SDK代码中以供应用程序操作存储桶。但是，如果这些承载着密钥的代码片段不慎泄露，比如开发者误将源码上传至公开仓库或者应用开发商在为客户提供的演示示例中未对自身SDK中凭据信息进行删除，这些场景将会导致对象存储凭据泄露，进而导致对象存储服务遭受入侵，攻击者通过冒用凭据所有者身份攻击对象存储服务。

3. 存储桶工具配置文件泄露

在对象存储服务使用过程中，为了方便用户操作存储桶，官方以及开源社区提供了大量的对象存储客户端工具以供用户使用，在使用这些工具时，首先需要在工具的配置文件或配置项中填写存储服务相关信息以及用户凭据，以便工具与存储服务之间的交互。在某些攻击场景下，例如开发者个人PC遭受钓鱼攻击、开发者对象存储客户端工具配置文件泄露等，这些编写在存储服务工具配置文件中的凭据以及存储桶信息将会被泄露出来，攻击者可以通过分析这些配置文件，从中获取凭据，而在这些工具中配置的，往往又是用户的云平台主API密钥，攻击者通过这些信息可以控制对象存储服务，在一些严重的场景，攻击者甚至可以控制用户的所有云上资产。

4. 前端直传功能获取凭据

在一些对象存储服务与Web开发以及移动开发相结合的场景中，开发者选择使用前端直传功能来操作对象存储服务，前端直传功能指的是利用iOS/Android/JavaScript等SDK通过前端直接向访问对象存储服务。前端直传功能，可以很好的节约后端服务器的带宽与负载，但为了实现此功能，需要开发者将凭据编写在前端代码中，虽然凭据存放于前端代码中，可以被攻击者轻易获取，但这并不代表此功能不安全，在使用此功能时，只要遵守安全的开发规范，则可以保证对象存储服务的安全：正确的做法是使用临时密钥而非永久密钥作为前端凭据，并且在生成临时密钥时按照最小权限原则进行配置。但是实际应用中，如果开发人员并未遵循安全开发原则，例如错误的使用了永久密钥，或为临时凭据配置了错误的权限，这将导致攻击者可以通过前端获取的凭据访问对象存储服务。攻击者通过分析前端代码，或者通过抓取流量的方式，获得这些错误配置生成的凭据，并以此发起攻击。

5. 云平台账号非法登录

云平台提供多种身份验证机制以供用户登录，包括手机验证、账号密码验证、邮箱验证等。在云平台登录环节，攻击者通过多种手法进行攻击以获取用户的登录权限，并冒用用户身份非法登录，具体的技术包括使用弱口令、使用用户泄露账号数据、骗取用户登录手机验证码、盗取用户登录账号等。攻击者使用获取到的账号信息进行非法登录云平台后，即可操作对象存储服务。

6. 实例元数据服务未授权访问

云服务器实例元数据服务是一种提供查询运行中的实例内元数据的服务，云服务器实例元数据服务运行在链路本地地址上，当实例向元数据服务发起请求时，该请求不会通过网络传输，但是如果云服务器上的应用存在RCE、SSRF等漏洞时，攻击者可以通过漏洞访问实例元数据服务。通过云服务器实例元数据服务查询，攻击者除了可以获取云服务器实例的一些属性之外，更重要的是可以获取与实例绑定的拥有操作对象存储服务的角色，并通过此角色获取对象存储服务的控制权。

4.5.3. 执行

1. 使用云API执行命令

攻击者利用初始访问阶段获取到的拥有操作对象存储服务的凭据后，可以通过向云平台API接口发送HTTP/HTTPS 请求，以实现与对象存储后台的交互操作。

对象存储服务提供了丰富的API接口以共用户使用，攻击者可以通过使用这些API接口并构造相应的参数，以此执行对应的对象存储服务操作指令，例如下载存储对象、删除存储对象以及更新存储对象等。

2. 使用对象存储工具执行

除了使用云API接口完成对象存储服务的执行命令操作之外，还可以选择使用对象存储工具来化简通过API接口使用对象存储服务的操作。

在配置完成存储桶信息以及凭据后，攻击者可以使用对象存储工具执行对象存储服务相应的操作名：通过执行简单的命令行指令，以实现对象存储桶中对象的批量上传、下载、删除等操作。

4.5.4. 持久化

1. 在存储对象中植入后门

针对对象存储服务的持久化攻击阶段，主要依赖于业务中采用的代码自动化部署服务将植入后门的代码自动部署完成。

在一些云上场景中，开发者使用云托管业务来管理其Web应用，云托管服务将使用者的业务代码存储于特定的存储桶中，并采用代码自动化部署服务在代码每次发生变更时都进行构建、测试和部署操作。

在这些场景中，攻击者可以在存储桶中存储的Web应用代码内安插后门代码或后门文件，并触发代码自动化部署服务将后门部署至服务器以完成持久化操作。这些存储着恶意后门将会持久的存在于Web应用代码中，当服务器代码迁移时，这些后门也将随着迁移到新的服务器上部署。

4.5.5. 权限提升

1. 通过Write Acl提权

对象存储服务访问控制列表（ACL）是与资源关联的一个指定被授权者和授予权限的列表，每个存储桶和对象都有与之关联的 ACL。

如果错误的授权给一个子用户操作存储桶ACL以及对象ACL的权限，即使该用户并未被赋予读取存储桶、写入存储桶、读取对象、写入对象的权限，这并不表示此用户不可以执行上述操作，该用户可以通过修改存储桶以及对象的ACL，将目标对象设置为任意读取权限，从而获取了存储桶以及存储对象的操作权限。因此，赋予子用户操作存储桶ACL以及对象ACL的权限，这个行为是极其危险的。

2. 通过访问管理提权

错误的授予云平台子账号过高的权限，也可能会导致子账号通过访问管理功能进行提权操作。

与通过Write Acl提权操作不同的是，由于错误的授予云平台子账号过高的操作访问管理功能的权限，子账号用户可以通过访问管理功能自行授权策略，例如授权QcloudCOSFullAccess策略，此策略授予子账号用户对对象存储服务全读写访问权限，而非单纯的修改存储桶以及存储对象的ACL。

通过此攻击手段，拥有操作对象存储服务权限的子账号，即使子账号自身对目标存储桶、存储对象无可读可写权限，子账号可以通过在访问管理中修改其对象存储服务的权限策略，越权操作存储桶中资源。

4.5.6. 窃取凭证

1. 云服务凭证泄露

在一些云上场景中，云服务会依托对象存储服务存储用户Web应用代码，用以自动化托管用户的Web应用程序。在这些场景中，用户的Web应用程序源码将会存储于存储桶中，并且默认以明文形式存储，在泄露的Web应用程序源码中，往往存在着Web应用开发者用来调用其他云上服务的凭据，甚至存在云平台主API密钥，攻击者可以通过分析泄露的Web应用程序源码来获取这些凭据。

2. 用户账号数据泄露

在一些场景中，开发者使用对象存储服务存储其业务中的用户数据，例如用户的姓名、身份证号码、电话等敏感数据，当然也会包含用户账号密码等凭据信息。

攻击者通过对存储桶中用户数据的提取与分析以窃取这些用户的凭据数据，并通过获取的信息进行后续的攻击。

4.5.7. 横向移动

1. 窃取云凭据横向移动

通过存储桶中Web应用程序源代码的分析，攻击者可能会从Web应用程序的配置文件中获取的应用开发者用来调用其他云上服务的凭据。攻击者利用获取到的云凭据，横向移动到用户的其他云上业务中。如果攻击者获取到的凭据为云平台主API密钥，攻击者可以通过此密钥横向移动到用户的所有云上资产中。

2. 窃取用户账号攻击其他应用

在一些场景中，开发者使用对象存储服务存储其业务中的用户数据，例如用户的姓名、身份证号码、电话等敏感数据，当然也会包含用户账号密码等凭据信息。

| 4.5.8. 影响

1. 窃取存储桶内项目源码

当开发者使用对象存储服务存储项目源码时，攻击者可以通过执行下载存储桶中的存储对象指令，获取到存储于存储桶中的项目源码，造成源码泄露事件发生，通过对源码的分析，攻击者可以获取更多的可利用信息。

2. 窃取存储桶内用户数据

当用户使用存储服务存储用户数据时，攻击者通过攻击存储服务，以窃取用户敏感数据，这些信息可能包含用户的姓名、证件号码、电话、账号信息等，当用户敏感信息泄露事件发生后，将会造成严重的影响。

3. 破坏存储数据

攻击者在获取存储桶操作权限之后，可能试图对存储桶中存储的数据进行删除或者覆盖，以破坏用户存储的对象数据。

4. 篡改存储数据

除了破坏存储服务中的用户数据之外，攻击者也可能对存储对象进行篡改操作，通过篡改存储桶中代码、文本内容、图片等对象以达到攻击效果。

在一个常见的场景中，用户使用对象存储服务部署静态网站，攻击者通过篡改其中页面内的文本内容以及图片，对目标站点造成不良的影响。

5. 植入后门

攻击者通过在对象存储服务中存储的Web应用代码中插入恶意代码，或者在项目目录中插入后门文件，当这些植入后门的Web应用代码被部署至云服务器时，攻击者可以利用这些后门发起进一步的攻击。当用户对存储桶中的Web应用代码进行迁移时，这些恶意代码也将随着业务代码一同迁移。

6. 拒绝服务

当攻击者拥有修改存储桶以及其中对象Acl访问控制列表时，攻击者可能会对存储对象的 Acl进行修改，将一些本应该公开访问的存储对象设置为私有读写，或者使一些本应有权限访问的角色无权访问存储对象。

攻击者可以通过此技术手段完成针对对象存储服务的拒绝服务攻击，从而影响目标资源的可用性。

4.6

云原生时代，腾讯如何做好容器安全体系建设？

| 4.6.1. 背景

容器安全作为云原生安全的重要组成部分，为用户业务的云原生落地提供了基础的安全保障。腾讯云凭借多年来在容器安全以及云原生安全领域的研究和实践运营经验，同时结合腾讯云容器平台 TKE 千万级核心规模容器集群治理经验，提出云原生容器安全体系框架，助力用户安全的实现云原生落地。

容器作为云原生重要的支撑技术，近年来被广泛的认可和应用。根据《中国云原生用户调查报告（2020）》显示，60%以上用户已在生产环境中应用容器技术，其中：43%的用户已将容器技术应用于核心生产环境，19%的用户已将容器技术用于非核心生产环境。

然而一次次安全事件的曝光，不管是特斯拉在亚马逊上的 Kubernetes 集群被入侵，还是 Docker Hub 频繁被爆含有漏洞和恶意程序的镜像，让用户在享受云原生红利的时候，产生了极大的安全担忧。根据云原生用户调查报告显示，容器的安全问题已成为用户应用云原生的最大担忧，其中63%的用户认为容器安全是紧迫的需求。

在实现云原生的主要技术中，容器作为支撑应用运行的重要载体，为应用的运行提供了隔离和封装，成为云原生应用的基础设施底座。因此容器的安全与否，将直接影响着整个云原生系统的安全性。

腾讯云依托在云原生以及安全方向的持续投入、积累和沉淀，一直致力于在云原生领域为用户提供更全面、更稳定、更安全的云原生服务，助力用户实现应用系统的云原生化，并且持续的保障其安全稳定的运行。

基于腾讯多年安全攻防技术的研究积累，持续在安全能力上的沉淀，以及对云原生安全领域的研究和实践运营，同时结合腾讯云容器平台 TKE 千万级核心规模容器集群治理经验，我们提出腾讯云原生容器安全体系框架，打造安全的容器云平台。框架描述了作为云原生安全底座的容器安全所覆盖的安全防护能力和范围，以及结合安全管理和安全运营，如何系统全面的保障容器安全。

| 4.6.2. 容器安全体系设计四大原则

云原生计算的特性决定了云原生容器安全在架构设计上，一个重要的原则就是安全能力的原生化，也就是在基础设施和基础架构上原生的提供安全能力，使得云原生应用能够做到上线即安全。

同时我们采用安全左移的思路，更早的投入安全资源和安全能力去更有效的收敛安全问题，然后将安全能力全面融入到 DevOps 体系中，实现面向 DevSecOps 的全生命周期安全防护。

容器安全防护的主体从主要以 IP 为标记的主机变成了以 Label、Tag 等作为标记的应用程序，其安全边界变的更加模糊。因此，需要采用零信任架构，通过全面有效的身份权限管理以及持续的检测与响应来实现对云原生应用的安全防护。



图49-安全体系设计原则

4.6.3. 全方位层次化的云原生容器安全体系框架

腾讯云容器平台的安全体系，采用层次化的方式，逐层实现安全防护。主要分为承载容器云平台的基础设施层安全、容器和容器云平台基础架构层安全、以及容器承载的应用层安全。

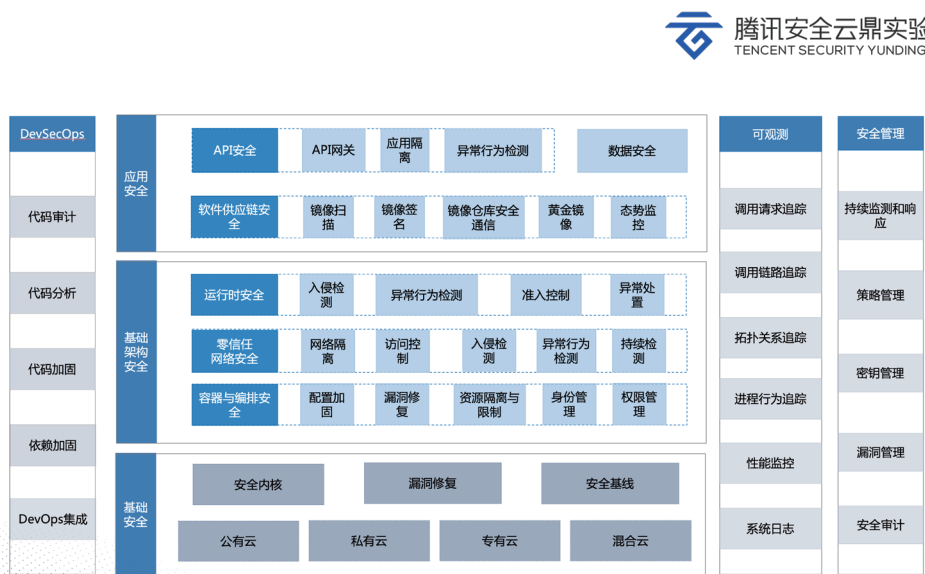


图50- 腾讯云容器平台安全体系

1. 基础设施安全

在基础设施层，不管是运行在公有云、私有云、专有云、还是混合云，我们会提供针对容器平台运行主机的安全防护措施，包括采用腾讯专有的安全内核和安全操作系统，以及对已知漏洞进行安全性的修复和管理。同时，针对主机上的基础软件，进行安全配置基线的检测与加固，在基础设施层面减小攻击面。

2. 基础架构安全

在容器云平台 TKE 的基础架构层，首先我们会提供针对容器和编排系统的安全防护措施，包括提供有效的资源隔离和限制措施，用户的身份和权限管理机制，基于 CIS 的安全配置加固，对云原生实现组件的漏洞管理与修复等。

其次，在网络安全上，我们采用零信任架构，持续的对集群网络进行监控和异常检测，保证网络的有效隔离和安全通信，包括租户、Service、Pod 等不同粒度的网络隔离，网络通信的加密，访问控制，网络的入侵检测，基于机器学习进行网络行为的异常检测与处置等。

最后，在容器运行前，首先我们会对启动的容器镜像进行准入检测，确保拉起的镜像符合基本的安全要求，从运行开始阶段进行安全风险的筛查；当容器启动运行后，我们会持续的对包括容器逃逸、反弹 shell、异常进程、文件篡改、高危系统调用等在内的入侵行为进行检测与告警，实现运行时的实时检测与防护。同时，我们还会结合机器学习、人工智能等大数据分析技术，对容器的行为进行监控、画像，从行为分析角度，对容器内的行为进行异常检测。多维度保障容器的运行安全。

3. 应用安全

在容器应用层，我们提供了全面的软件供应链安全防护措施，通过提供对镜像的安全扫描能力，深度扫描本地/仓库镜像，分析镜像的安全漏洞、恶意文件、敏感信息等。通过多维度深度扫描，帮助用户发现不安全镜像，从源头上解决容器镜像安全问题。

同时我们还提供针对容器镜像的安全态势监控能力，持续监测镜像的安全态势。另外，我们还可以通过 API 网关、API 监控等方式，对微服务间的 API 通信进行安全防护，包括 API 的访问控制、身份管理，调用链异常分析等。

4.6.4. 安全能力全面原生化

云原生依托容器、服务网格等关键技术，实现了更轻量的隔离方式、更灵活的负载管理、更复杂的容器网络、更短的容器生命周期、以及更敏捷的开发流程。这些重要变化，使得传统的安全防护手段很难发挥应有的作用。

因此，要实现容器安全或者是云原生安全，一定要充分匹配云原生的特性，采用原生安全的方式。这里的原生安全包括两个方面：一方面是原生的基础安全，我们在容器、编排、微服务等云原生基础架构的构建过程中，就充分的融入安全性的设计和考虑，使我们的云原生系统天然的就具备安全特性，使得安全成为像计算、存储、网络一样的基础能力；另一方面，就是安全能力的云原生实现，充分利用云原生的技术优势，来实现安全检测与防护能力，使相关的安全能力同样具备云原生的低成本、高效率和高可用特性。

4.6.5. 安全防护全生命周期

在云原生架构中，业务的快速迭代以及容器生命周期短、业务复杂、网络复杂等特点会造成运行时的安全检测投入成本很高。因此，基于上述层次化的安全框架，我们在容器生命周期的各个阶段，嵌入相应的安全能力，实现对容器的全生命周期安全防护。

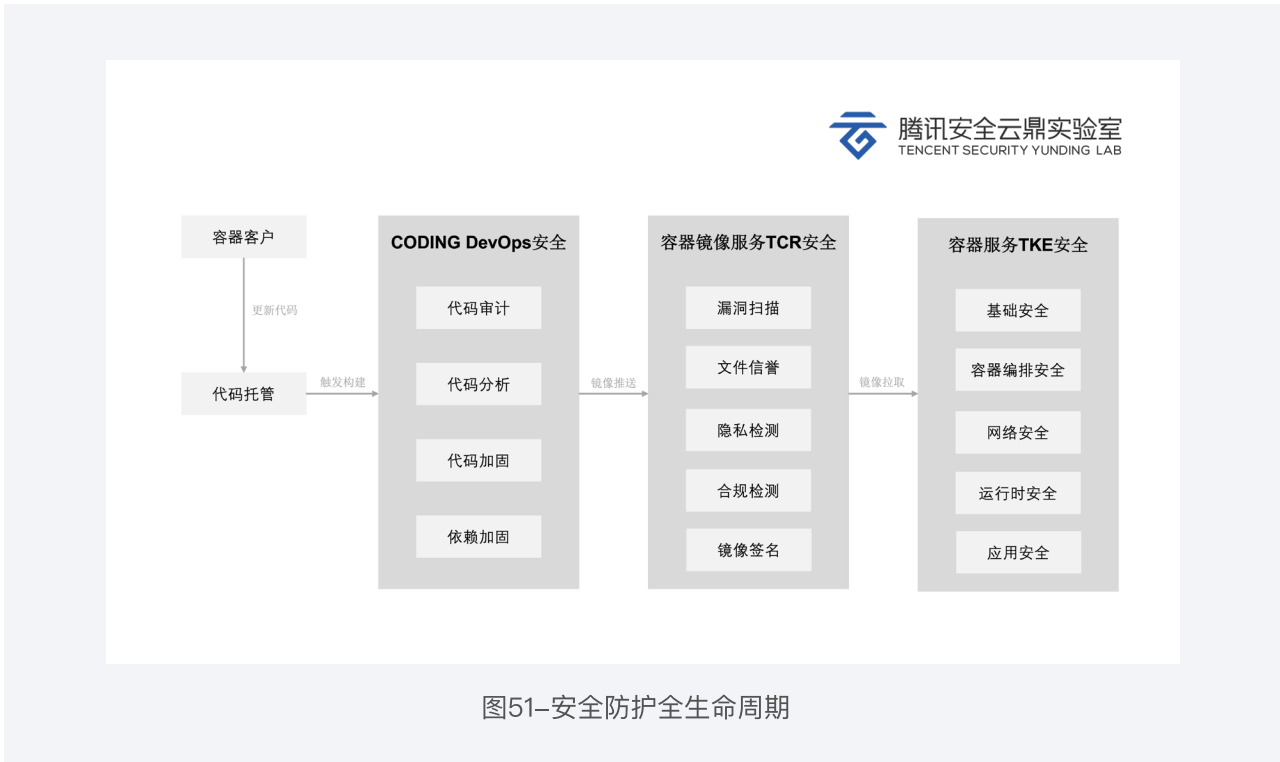


图51-安全防护全生命周期

全生命周期防护的一个重要的特点就是实现“安全前置”或者“安全左移”，在软件开发生命周期的更早阶段，投入更多的安全资源和能力，嵌入安全动作，来有效的收敛安全漏洞问题，尽可能更早的确定其安全性。

容器安全框架有效的与腾讯云 CODING 产品进行协同联动，在 DevOps 流程中嵌入安全能力，通过对应用程序的代码进行安全分析、审计、加固等措施，在开发阶段保障代码的安全性。

同时，在容器镜像服务 TCR 以及容器服务 TKE 中，分别嵌入镜像安全以及容器运行安全的相关安全能力，实现全生命周期的安全防护，打通 DevSecOps 闭环。

4.6.6. 安全威胁全面可观测

容器化的基础设施使得应用自身变的更快、更轻，一台主机上可以快速部署运行几十个、甚至上百个容器，应用的部署密度以及变化频率，较传统环境，有着巨大的变化。

正所谓“未知攻焉知防”，面对云原生架构下的大规模集群以及海量灵活的微服务应用，只有知道集群中应用的具体操作、行为，才能够进行高效的安全防护。

腾讯云容器平台 TKE 提供了全面的可观测服务，这既包括了系统日志、应用日志等一整套完善的日志管理服务，还包括了各类运行指标的高性能监控服务，以及进程行为追踪、服务调用链追踪等追踪服务，使我们能够清晰的了解系统的详细的运行状况，进而快速高效的进行威胁的检测、追踪与溯源。

除此之外，我们还提供了对安全事件和安全告警的全面可观测服务，使得安全威胁更加直观可视化，能够快速进行的进行异常定位，快速的实现应急响应。

4.6.7. 安全责任共担

腾讯云在安全建设上，一直倡导和践行着安全责任共担的原则，对于容器云平台的安全性保障，同样需要腾讯云以及云上用户的共同努力。

腾讯云的责任主要包括保护硬件、内核、操作系统等基础设施环境的安全，同时还包括保护容器、集群等容器云基础架构的安全，例如安全的配置和加固、身份权限的管理、网络的隔离以及入侵检测等。

客户的责任主要是管理和维护容器云上的工作负载，比如应用的代码安全、容器镜像的安全、相关版本的升级以及对监控、告警的响应等。

4.6.8. 总结

云原生架构给安全防护带来了前所未有的挑战，容器安全作为云原生安全的技术底座，对云原生安全起到了重要的支撑作用。

在云原生、安全左移、零信任等设计原则的指导下，我们提出层次化的云原生容器安全体系框架，从基础设施安全到容器基础架构安全，再到容器化应用的安全，逐层实现安全防护。同时，容器安全体系还与 DevOps 体系进行协同联动，在 DevOps 流程中嵌入安全能力，实现安全左移，降低运行过程中安全检测和防护的成本。在安全管理和运营上，通过密钥管理、安全策略管理、漏洞管理等服务，实现对用户云上的容器服务持续的检测和响应，确保其安全性。我们希望通过这一套完备的安全防护体系，打造一个原生安全的容器云平台，助力用户安全的实现云原生转型，享受云原生带来的红利。